Untying Prefill and Decode Loops in Looped Language Models

Yang Hu

Abstract

Looped language models reuse one block of layers several times per token, and they run the same number of loops when they prefill the prompt and when they decode each answer token. The two phases are not alike: prefill runs once and in parallel, decoding runs again for every generated token. We untie them. A small *loop adapter*, trained once on a frozen looped model, lets it prefill with m loops and decode with n, for all 16 pairs up to four loops; each decode loop reads the prompt KV of a matching prefill loop through a learned map, or, to save memory, all read one stored cache. The resulting map has a clear shape: decode cheaply, prefill deeply. On Ouro-2.6B with one decode loop, prefilling with four loops raises the average over 11 benchmarks from 44.2 to 56.8 (49.8 for native 1/1 distilled on the same data), prefilling with two to four loops is best on 14 of 15 settings, and with as many decode as prefill loops the adapter matches the native model. On Ouro-2.6B-Thinking, with one stored cache, prefill 4 / decode 2 matches the average accuracy of native 4/4 over eight benchmarks at $3.2\times$ its decode throughput, and answers in $0.59\times$ the time at batch 1. Looped decision models follow the same rule from both sides: more prefill than decision loops helps on 12 of 13 pairs, and fewer hurts on all 13. Loops are best spent where they are paid once. Code, results and per-item outputs: `https://github.com/yanghu819/untying-prefill-decode`.

1 Introduction

Looped language models such as Ouro (Zhu et al., 2025), Huginn (Geiping et al., 2025) and Mixture-of-Recursions (Bae et al., 2025b) obtain depth without adding parameters by running a shared block of layers several times per token (Dehghani et al., 2018). Under the fixed-depth inference we study, the price is paid twice: every generated token runs every loop, and every loop keeps its own key–value (KV) cache.

Yet the two phases of inference are not alike. Prefill is parallel and paid once per request; decoding is sequential and paid for every token. A looped model nevertheless runs the same number of loops in both. What do prefill loops buy, and what do decode loops buy? If the answers differ, the two counts should be chosen separately.

We write prefill m / decode n for prefilling the prompt with m loops and decoding every answer token with n loops, and native n/n for the unmodified model, which uses n loops for both. When $m \neq n$, the decode loops must attend to prompt KV produced by a different loop than their own. Our *loop adapter* combines LoRA on the frozen model with a learned map on the prompt KV for every decode loop (Figure 2): decode loop k reads prefill loop $m - n + k$ (aligned caches), or every decode loop reads one stored prompt cache. We train one loop adapter and evaluate it at all 16 pairs (m, n) with its parameters fixed.

The resulting map (Figure 1) has a clear shape: decode cheaply, prefill deeply. With one decode loop the best prefill count is two to four on 14 of 15 benchmark settings; within the same adapter, prefilling with three or four loops instead of one raises MMLU from 51.4 to 66.6 and HumanEval+ from 51.2 to 65.9. A native model can only move along the diagonal of this map; the adapter opens the whole square. With prefill 4 the average over 11 benchmarks rises from 44.2 to 56.4 at one decode loop and from 63.0 to 66.4 at two, above native models distilled on the same data (Table 1); without the adapter the same untied configuration collapses.

Untying pays off in time. On Ouro-2.6B-Thinking, prefill 4 / decode 2 stays within two points of native 4/4 on six of eight benchmarks (Figure 4); averaged over them it matches native 4/4 at $3.2\times$ its decode throughput (Figure 6) and answers 192 timed GSM8K problems in $0.59\times$ the

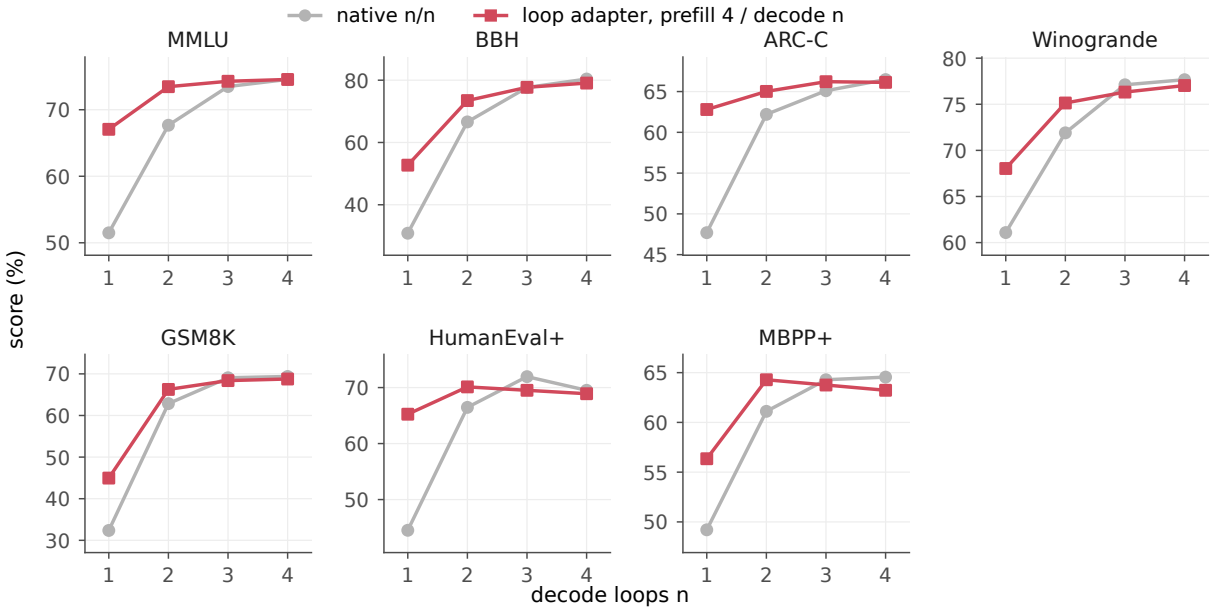


Figure 1: **Decode cheaply, prefill deeply.** Ouro-2.6B, accuracy (%) against the number of decode loops n. Grey: the native model, which runs n loops for both prefill and decode (n/n). Red: one loop adapter (aligned caches) that prefills with four loops and decodes with n. With few decode loops, four prefill loops recover much of the accuracy of the deeper native model; with three or four it matches native. Each panel uses the items all points share; all settings and every (m, n) cell: Appendix E.

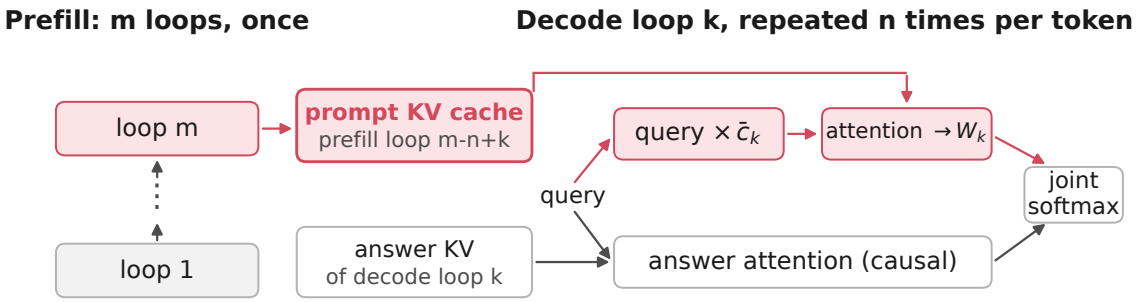


Figure 2: **The loop adapter.** The frozen model prefills the prompt with m loops. Each of the n decode loops reads the prompt KV of prefill loop $m - n + k$ (aligned caches) through its own map: a learned scale on each rotary (RoPE) pair of the keys, which can be moved onto the query so the cache is never rewritten, and a value map applied after attention. Attention over the prompt and over the generated tokens is computed separately and combined into one softmax (joint softmax), as in blockwise (FlashAttention-style) attention. With one stored cache instead, every decode loop reads loop m and the prompt KV is stored once for every n: total KV $b(\ell + n\,t)$ instead of $b\,n(\ell + t)$, with b bytes per token and loop and ℓ, t the prompt and answer length.

time at batch 1: the decode loops set the cost, and the prefill loops buy back the accuracy.

The same rule holds in looped decision models, which score options over a long context without generating text (Section 4.5), and there we see it from both sides: prefilling the context with more loops than the decision uses is significantly above a matched-training control on 12 of 13 (m, n) pairs, and fewer prefill loops are below it on all 13, by up to 13 points, with the same direction in each of four training seeds.

Our contributions are: (i) the observation that prefill loops and decode loops buy different things, measured on the full $m \times n$ map of two looped language models and of looped decision models; (ii) a loop adapter that unties them on existing checkpoints, with aligned prompt caches that match the native model at equal depth or with one exact prompt KV cache that stores $b(\ell+n\,t)$ bytes of KV instead of $b\,n(\ell + t)$; and (iii) measured savings: native-4/4 accuracy at $3.2\times$ the decode throughput, and $0.59\times$ the answer time at batch 1. Code,

per-item results and a script that recomputes every statistic are released.

2 Related Work

Looped and elastic depth. Weight-tied depth recurrence goes back to Universal Transformers and cross-layer sharing (Dehghani et al., 2018; Lan et al., 2020); looped transformers are computationally universal (Giannou et al., 2023), can simulate chain-of-thought steps in latent space (Saunshi et al., 2025) and generalise in length with adaptive step counts (Fan et al., 2025b), a vertical counterpart of continuous latent reasoning (Hao et al., 2025). Relaxed Recursive Transformers (Bae et al., 2025a) convert pretrained models into looped ones with per-loop LoRA; Mixture-of-Recursions (Bae et al., 2025b) and Huginn (Geiping et al., 2025) route or scale the loop count. Accordion (Zhang et al., 2026) and Elastic Latent Recurrence (Ladhak et al., 2026) train one model for several depth budgets; ELR also finds that keeping only the last loop's KV retains most of the gain. The Parallel Loop Transformer (Wu et al., 2025a), LoopCoder-v2 (Yang et al., 2026) and shared-memory looped transformers (Monea et al., 2026) let later loops read the first loop's KV plus a short local window. All run the same number of loops for prefill and decode. Ouro (Zhu et al., 2025) reuses the last loop's KV for generated tokens but loses more than 10 GSM8K points when prefill does the same; our adapter is trained to read prompt KV across loops. FlashLoop (Yang and Liu, 2026) speeds up looped models without training through token-sparse loop updates, sparse attention and KV-residual quantisation.

Adaptive and phase-specific computation. Halting (Graves, 2016; Banino et al., 2021), token-level early exit (Xin et al., 2020; Elbayad et al., 2020; Schuster et al., 2022; Elhoushi et al., 2024) and token routing (Raposo et al., 2024) spend depth per token; CALM must also fill the skipped layers' KV for later tokens, the problem our maps solve per loop. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) pairs a cheap drafter with a deep verifier, and pause tokens add compute at prompt positions (Goyal et al., 2024). Continuous Depth Batching (Schwethelm et al., 2026) prefills at full depth, exits early during decode and studies caches shared across depths; Position-Aware Depth Decay (Fan et al., 2025a) skips more layers for later tokens; OverFill (Kim et al., 2025) decodes with a width-pruned model; Fixed Points Part II (Huang et al., 2026) distils a light prefill network; POP (He et al., 2026) and SPEED (Oh et al., 2026) prefill shallower than they decode; Decode-Branch Transformers (Liu et al., 2026), PHD (Wu et al., 2025b) and State-Prediction Separation (Langford et al., 2026) add decode-side computation. Serving systems already run the two phases on separate hardware (Zhong et al., 2024; Patel et al., 2024; Qin et al., 2025) or co-schedule them (Agrawal et al., 2024; Kwon et al., 2023); untying the loops lets each phase also use its own depth. We vary prefill and decode loops independently within one loop adapter and measure what each buys.

Asymmetric KV producers and consumers. YOCO (Sun et al., 2024) has a self-decoder produce one global KV cache that a cross-decoder reads; YOCO-U (Sun et al., 2026) loops the self-decoder, keeping KV storage independent of the loop count while every token still runs the loops; KITE (Hu et al., 2026) expands a checkpoint into two towers so that bulk prefill runs only the KV-producing tower. These architectures are built for the split; we adapt an existing looped model after pretraining and choose the prefill loops m and decode loops n per request.

Sharing and transferring state. KV is shared across heads (Shazeer, 2019; Ainslie et al., 2023) and layers (Brandon et al., 2024; Zuhri et al., 2025), and evicted or compressed per token (Xiao et al., 2024; Zhang et al., 2023; Li et al., 2024; Cai et al., 2024), which is orthogonal to and composable with our per-loop reads. MELT (Vendrell et al., 2026) post-trains Ouro to keep one gated KV cache per layer shared by all loops plus a small per-loop sliding-window cache, decoupling memory from depth; it does not separate prefill from decode loops or map which depths a task needs. Looped Latent Attention (Neill and Reid, 2026) compresses KV across loops; BeNice! (Prairie et al., 2026) and LSTM-style looped transformers (Kirsch, 2026) share it; Cache-to-Cache (Fu et al., 2025), HeteroFold (Yun et al., 2026), Swarm Telepathy (Alabdulkareem et al., 2026) and CacheLink (Qiu et al., 2026) transfer KV between models, and BellowsKV (Zhang and Chen, 2026) shares one latent across layers. Like CacheLink, we train our maps through the receiver's output; within one model tokens, layers and posi-

tions align, so a RoPE-commuting scale and a value map suffice. For hybrid attention / linear-recurrence models (Yang et al., 2024; Dong et al., 2024), state-swap interventions (Afendulev et al., 2026) and LatentPort (Villani, 2026) study the two kinds of state; Appendix K relates them to prefill loops.

3 Method

We describe the loop adapter for Ouro; SenseNova-Loop-9B uses a lighter variant without maps (Appendix A).

Prefill and decode. "prefill m / decode n" prefills the prompt with m loops and decodes every answer token with n loops; native n/n is the unmodified model. The frozen model prefills the prompt with m loops and keeps only the last loop's prompt keys and values. The decoder is the same frozen model with rank-16 LoRA (Hu et al., 2021) on its attention projections, running n loops per token.

RoPE-commuting maps. Decode loop k reads the prompt KV cache through a map per (loop, layer, KV head). The key map is one complex scale c_j per rotary frequency pair j (Su et al., 2021): viewing the pair of key coordinates (k_{2j}, k_{2j+1}) as a 2-D vector, $c_j = r\,e^{i\phi}$ stretches it by r and turns it by ϕ. Writing a key pair rotated for position s as a complex number $\tilde{k}_j = e^{is\theta_j}k_j$, scaling commutes with the rotation, $c_j\tilde{k}_j = e^{is\theta_j}(c_j k_j)$, and $\mathrm{Re}\langle q_j, c_j\tilde{k}_j\rangle = \mathrm{Re}\langle \bar{c}_j q_j, \tilde{k}_j\rangle$: the map is applied to the query and the stored cache is never rewritten. The value map is a full linear map W_k per head; because attention is linear in the values, $\sum_i a_i W_k v_i = W_k \sum_i a_i v_i$ is applied once to the attention output over the prompt. Prompt and generated tokens share one softmax: the two parts are computed by two attention calls with outputs o_P, o_A and log-sum-exps λ_P, λ_A of their attention logits, and merged as $g\,o_P + (1-g)\,o_A$ with $g = \sigma(\lambda_P - \lambda_A)$, which equals one softmax over both (the blockwise softmax of FlashAttention (Dao, 2023)). The prompt KV is therefore stored once for every n, and total KV is $b(\ell + n\,t)$ instead of $b\,n(\ell + t)$. Serving from one cache equals materialising a mapped prompt cache per loop; in bfloat16 all 512 checked greedy tokens agree.

Aligned caches. With one cache, every decode loop reads the last prefill loop. The aligned variant keeps one prompt cache per decode loop and lets decode loop k read prefill loop $m - n + k$ through its own map, so at $m = n$ every decode loop reads exactly the KV the native model would. It stores the prompt n times, as native n/n does, and is trained identically. It is the more accurate layout, most of all on long-context retrieval; the single cache is the smaller one.

What the maps do. With the same training and the maps frozen at the identity, the decoder is within 0.4 points on MMLU (significantly lower at 1/1 and 2/2, by 0.4 and 0.3) has no significantly negative GSM8K difference (at 4/1 it scores 2.3 points higher $_{[0.2,\ 4.5]}$), but loses 11.4 points on RULER-16K at prefill 4 / decode 1 (Appendix I): in these ablations the maps matter for long prompts.

Training. We distil (Hinton et al., 2015): we minimise $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ over answer positions, with the native 4/4 model as teacher, on 48M answer tokens of reasoning traces, instruction data, long-context data and web text. One (m, n) pair is sampled per micro-batch, so one set of parameters (63M, 2.4% of Ouro-2.6B) serves all 16 pairs. Details are in Appendix A.

4 Experiments

Setup. Models: Ouro-2.6B, Ouro-1.4B and their Thinking variants (Zhu et al., 2025) (four loops) and SenseNova-Loop-9B-Preview, a hybrid of Gated-DeltaNet and full-attention layers whose middle block is visited twice. Benchmarks cover knowledge and commonsense (MMLU (Hendrycks et al., 2020), MMLU-Pro (Wang et al., 2024), ARC-Challenge (Clark et al., 2018), GPQA-Diamond (Rein et al., 2023), HellaSwag (Zellers et al., 2019), Winogrande (Sakaguchi et al., 2020)), long-context retrieval (RULER (Hsieh et al., 2024) at 4K/8K/16K, HELMET (Yen et al., 2024)), reasoning and instruction following (BBH (Suzgun et al., 2022), IFEval (Zhou et al., 2023)), math (GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021; Lightman et al., 2023)) and code (HumanEval+ and MBPP+ (Chen et al., 2021; Liu et al., 2023)). We follow the base-model protocol of the Ouro models where it is available (shots, chain of thought, strict-match scoring for GSM8K, final-answer scoring for MATH-500, evalplus for code), with documented

```
# blocks: frozen looped block, L layers; m/n: prefill/decode loops
# c[k][l] = 1+a+1j*b per RoPE pair; W[k][l] = I+delta (init: I)
# LoRA on q, k, v, o is active only while decoding.

def prefill(prompt, m): # prompt minus its last token
    h = embed(prompt)
    for j in range(m):
        for l in range(L):
            h, K[j][l], V[j][l] = blocks[l](h) # every loop's K, V
    return K, V

def attend(q, K, V, kv_answer, c, W): # layer l, decode loop k
    qc = complex_mul(q, c.conj()) # key scale moved to the query
    o_p, lse_p = flash_attn(qc, K, V) # prompt part
    o_p = o_p @ W.T # attention is linear in V
    o_a, lse_a = flash_attn(q, *kv_answer, causal=True)
    g = torch.sigmoid(lse_p - lse_a) # exact softmax over both
    return g * o_p + (1 - g) * o_a

def decode(token, K, V, m, n): # last prompt token, then answer
    h = embed(token)
    for k in range(n):
        j = max(m - n + k, 0) # aligned; one cache: j = m - 1
        for l in range(L):
            h = blocks[l](h, attn=lambda q, kv: attend(
                    q, K[j][l], V[j][l], kv, c[k][l], W[k][l]))
    return lm_head(h)
```

```
# training: one (m, n) per micro-batch,
# teacher = the native 4/4 model
m, n = torch.randint(1, 5, (2,)).tolist()
with torch.no_grad():
    t = native(x, loops=4) # frozen, no adapter
s = model(x, prefill=m, decode=n) # LoRA + maps
kl = kl_div(s.log_softmax(-1),
            t.log_softmax(-1), log_target=True,
            reduction="none").sum(-1)
loss = kl[answer].sum() / (answer.sum() - len(x))
```

Figure 3: **The loop adapter in PyTorch-style pseudocode.** Decode loop k reads prefill loop $m - n + k$ (loops counted from zero here; the one-cache variant keeps only the last prefill loop) through its own key scale (applied to the query) and value map (applied to the attention output), and the prompt and answer attention are combined exactly through their log-sum-exp.

substitutions; protocols, set sizes and the effect of departing from them are in Appendix B. One loop adapter per model is trained once; on Ouro-2.6B and Ouro-1.4B it is evaluated at all 16 pairs, on the Thinking models at the pairs reported, and on SenseNova-Loop-9B at prefill 1 and 2 with one decode visit. We compare with the native model at the same number of decode loops. Every comparison is paired over items with 95% bootstrap intervals.

4.1 The task map

Figure 1 plots accuracy against decode loops. The native model loses most of its accuracy when it decodes with fewer loops; the loop adapter, prefilling with four loops, keeps much of it: at one decode loop MMLU rises from 51.5 to 67.1, BBH from 30.9 to 52.7 and HumanEval+ from 44.5 to 65.2. Decode loops matter most for multi-step math (native GSM8K: 32.4 at one loop, 69.4 at four), and the adapter adds 13 points there at one decode loop. With three or four decode loops the adapter tracks the native model, also on long-context retrieval (RULER-16K at 4/3: 79.8 against 78.7 for native 3/3). The full $m \times n$ map is in Appendix E.

4.2 Main results

Table 1 compares fixed configurations at the same number of decode loops. With one decode loop, prefilling with four loops raises the average from 44.2 (native 1/1) to 56.8, above native 1/1 distilled on the same data (49.8); with two, the aligned adapter averages 67.2, against 63.0 for native 2/2 and 65.3 for its distilled version, within 3.0 points of native 4/4 with half the decode loops. The one-cache variant, at a fraction of the prompt memory, gives up under one point (56.4 and 66.4). The adapter is what makes untying work: without it the same untied configurations average 26.6 and 30.9, below even native 1/1 (on GSM8K at 4/2, 11.7 against 66.7 with the adapter). Within the same adapter, deeper prefill alone adds up to 15 points (Table 6), so the gain is not only extra training. Choosing the prefill count per benchmark on held-out items instead of fixing it at four beats native 1/1 significantly on 14 of 15 settings (Appendix D), and Ouro-1.4B shows the same pattern (Appendix F). Equal decode loops do not mean equal latency (Section 4.4).

Training cost. The adapter needs little data (Table 14): trained on 6M answer tokens, an eighth of the default budget, it already averages 53.7 and 65.2 at prefill 4 / decode 1 and 2, against 56.4 and

n	method	MMLU	MMLU-Pro	BBH	ARC-C	HellaSw.	Winogr.	GSM8K	MATH	HE+	MBPP+	RULER-16K	Avg.
1	Native 1/1	51.5	26.9	30.9	47.7	69.0	61.1	32.4	32.2	44.5	49.2	41.4	44.2
1	Training-free 4/1	61.9	0.0	0.8	37.5	40.4	51.9	8.9	4.6	29.3	48.9	8.0	26.6
1	LoRA distill. 1/1	52.7	26.7	40.4	55.0	70.1	66.5	40.4	**37.0**	54.3	50.3	54.7	49.8
1	Loop adapter, one cache 4/1	66.6	**47.4**	51.7	62.5	72.4	67.1	44.7	25.8	64.6	55.3	62.8	56.4
1	**Loop adapter, aligned 4/1**	**67.1**	45.0	**52.7**	**62.8**	**72.7**	**68.0**	**45.0**	25.4	**65.2**	**56.4**	**64.7**	**56.8**
2	Native 2/2	67.7	45.0	66.6	62.2	77.8	71.9	62.9	**50.6**	66.5	61.1	60.8	63.0
2	Training-free 4/2	71.3	4.0	4.0	41.0	50.6	65.5	11.7	0.0	7.3	33.1	52.0	30.9
2	LoRA distill. 2/2	67.2	46.2	70.5	62.9	77.6	74.6	**69.6**	46.8	69.5	63.0	70.3	65.3
2	Loop adapter, one cache 4/2	73.3	51.2	71.2	**65.1**	78.0	**75.1**	66.7	47.0	68.3	62.7	72.2	66.4
2	**Loop adapter, aligned 4/2**	**73.5**	**54.0**	**73.5**	65.0	**78.2**	**75.1**	66.3	45.8	**70.1**	**64.3**	**73.5**	**67.2**
4	Native 4/4	74.5	55.2	80.4	66.5	79.5	77.7	69.4	53.6	69.5	64.5	81.3	70.2

Table 1: **Main results.** Ouro-2.6B, accuracy (%) grouped by the number of decode loops n; "4/n" prefills with four loops. Native: the unmodified model. Training-free: the frozen model prefills with four loops and decodes from the last loop's prompt KV without an adapter. LoRA distill.: native n/n distilled (Hinton et al., 2015) with LoRA (Hu et al., 2021) on the same data, teacher (native 4/4) and tokens as the loop adapter. Bold: best in each group. Each column uses the items all rows share; MMLU and GSM8K are the full sets. One cache: all decode loops read the last prefill loop's cache; aligned: decode loop k reads prefill loop $m - n + k$ (Section 3). Training seed 0; over three training seeds the one-cache average is 56.4 ± 0.0 at 4/1 and 66.6 ± 0.2 at 4/2 (mean $\pm$ s.d.). Paired intervals and the per-benchmark choice of m: Appendix D.

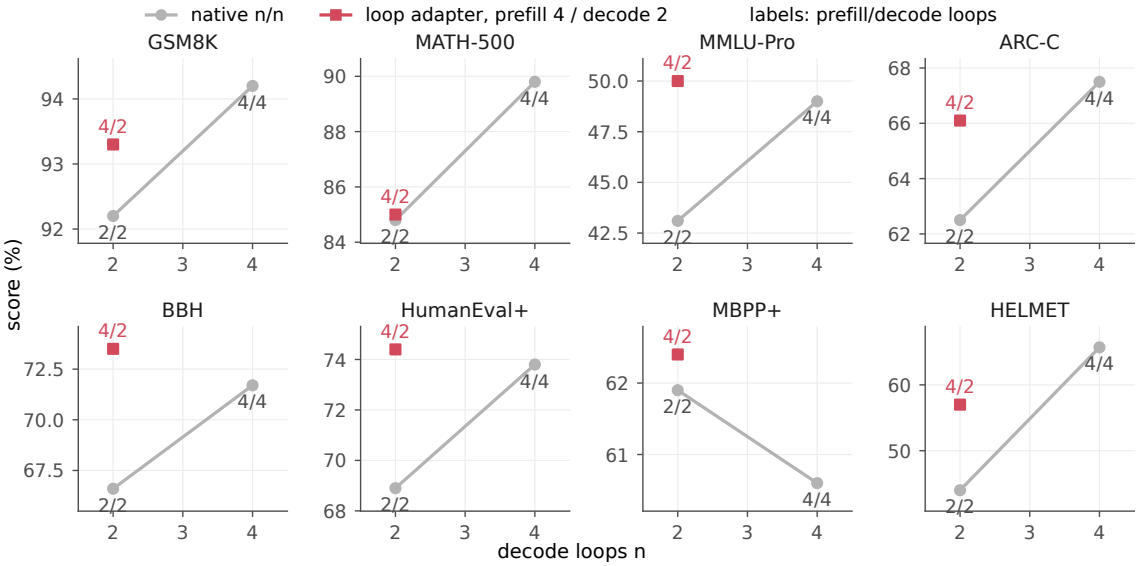


Figure 4: **Prefill 4 / decode 2: two decode loops, point estimates within two points of four on six of eight benchmarks.** Ouro-2.6B-Thinking, score against the number of decode loops. Grey: native 2/2 and 4/4. Red: the loop adapter (one cache) with prefill 4 / decode 2. Same items for all three in each panel; HELMET is averaged over its categories. Table 10 gives every score, the paired intervals and the same adapter at prefill 2 / decode 2.

66.4 with 48M, and neither a smaller nor a larger LoRA rank changes the result.

4.3 More prefill loops, fewer decode loops

Figure 4 applies the finding to a reasoning model, Ouro-2.6B-Thinking, at prefill 4 / decode 2. With the same number of decode loops as native 2/2, prefill 4 / decode 2 scores higher on all eight benchmarks, by up to 13 points on HELMET and 7 on MMLU-Pro and BBH; against native 4/4, which decodes with twice the loops, it is within two points on six of the eight. The prefill loops themselves carry the gain: the same adapter at prefill 2 / decode 2 is up to 8 points lower on MMLU-Pro and HELMET, and on Ouro-1.4B-Thinking four prefill loops instead of two are better on six of eight benchmarks. prefill 4 / decode 2 also scores above native 2/2 distilled on the same data on all eight (Appendix G).

Against measured decode throughput (Figure 5), prefill 4 / decode n is more accurate than native n/n at one or two decode loops and has

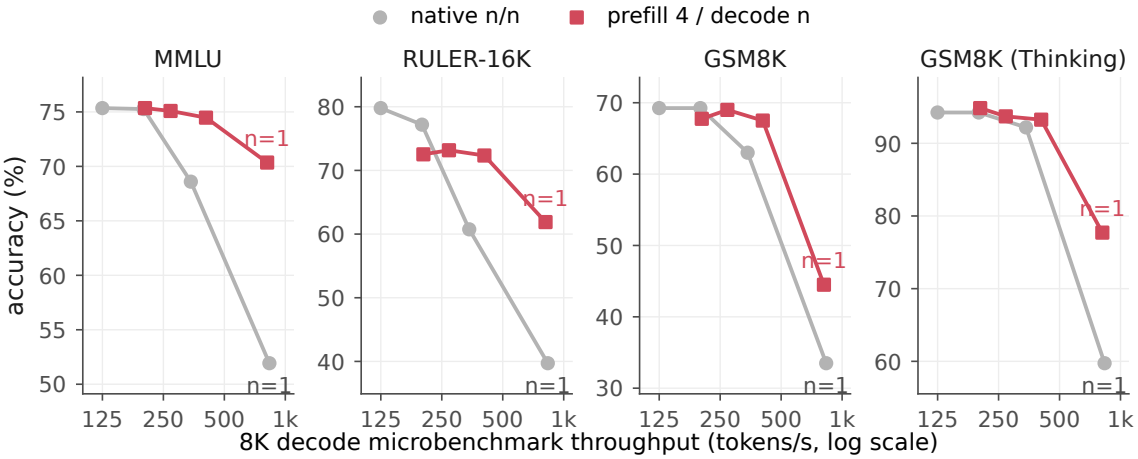


Figure 5: **At one or two decode loops, the adapter is more accurate than native n/n.** Native n/n (grey) and the loop adapter with prefill 4 / decode n (red, one cache); each line runs from $n = 1$ (labelled, right) to $n = 4$ (left; Thinking adapter $n = 1$–3). Throughput is a separate decode microbenchmark: best decode tokens/s over batch sizes at 8K context on one B300 (same for the base and Thinking models, which share the architecture); one stored prompt KV cache admits larger batches as n grows. Ouro-2.6B on MMLU (20 questions per subject), RULER-16K (260 items) and GSM8K (400 problems); Ouro-2.6B-Thinking on all 1,319 GSM8K problems.

training tokens	LoRA rank	4/1 avg.	4/2 avg.
6M	16	53.7	65.2
12M	16	55.5	65.9
24M	16	55.7	66.1
48M (default)	16	56.4	66.4
12M	4	55.4	65.8
12M	64	55.3	66.5

Table 2: **The loop adapter is cheap to train.** Ouro-2.6B, average over the benchmarks of Table 1 (each on the items all runs share) at prefill 4 / decode 1 and 2, for adapters trained on fewer answer tokens of the same mixture and with other LoRA ranks.

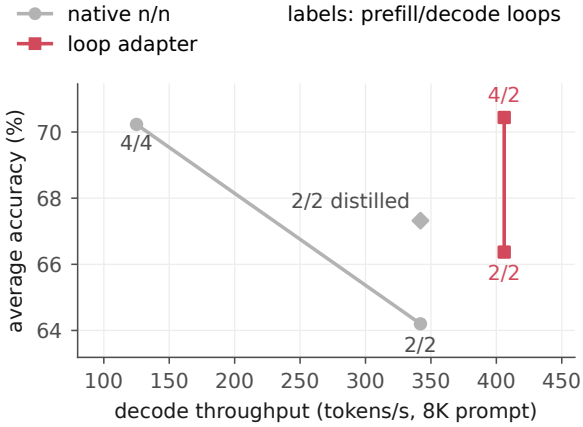


Figure 6: **Prefill 4 / decode 2 matches native 4/4 at 3.2× its decode throughput.** Ouro-2.6B-Thinking: average accuracy over the eight benchmarks of Figure 4 (each on the items all five configurations share) against decode throughput (best tokens/s over batch sizes at an 8K-token prompt, Table 13). Labels: prefill/decode loops. The adapter decodes at the same cost at 2/2 and 4/2, and distilled native 2/2 (LoRA merged) at that of native 2/2; the four prefill loops add 4.0 points.

higher throughput at three or four, because the one stored prompt cache admits larger batches; prefill 4 / decode 2 runs at 1.19× the throughput of native 2/2 and 3.2× that of native 4/4. The gain grows with the prompt: against native 4/4 it is 2.4× at a 2K-token prompt and 3.3× at 15K, where prefill 4 / decode 2 also outpaces native 2/2 (1.12×). With aligned caches the adapter decodes at about the cost of native n/n, storing the prompt n times (314 against 342 tokens/s at $n = 2$, still 2.5× native 4/4); the single cache buys the remaining speed.

4.4 End-to-end time

We time whole answers at batch 1, prefill included (Table 12 in Appendix H). On Ouro-2.6B-Thinking, prefill 4 / decode 2 answers 192 timed GSM8K problems in 0.59× [0.52, 0.66] the time of native 4/4. On all 1,319 problems it scores 93.3 against 94.2, a paired difference of −1.0 [−2.0, 0.1]. Prefilling with 4 rather than 2 loops before decoding with two changes the answer time by 1.02× [0.92, 1.13], and prefill 4 / decode 2 takes 1.20× [1.05, 1.38] the time of the distilled native 2/2, at similar accuracy on these problems: the saving comes from decoding with two loops, and what the extra prefill loops add is the accuracy of Sec-

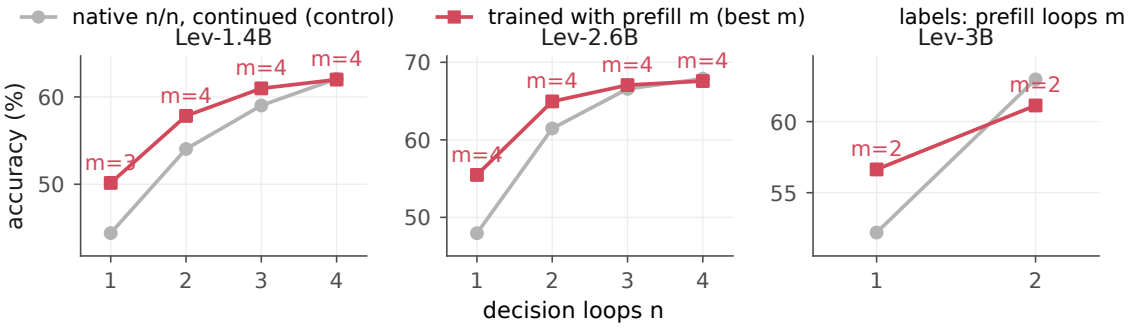


Figure 7: **Decision models gain when the context gets more loops than the decision.** Lev decision models (Song et al., 2026), LevBench development set (3,270 questions), accuracy against decision loops n. Grey: native n/n continued with the same data and updates. Red: the best of the models trained to prefill with m loops (labels); seed 0. All pairs: Table 15.

tion 4.3. On MATH-500 the saving is smaller: $0.63\times$ the time of native 4/4 but $1.22\times$ that of native 2/2 (Appendix H). A smaller model is not the same trade: native Ouro-1.4B-Thinking 4/4 takes about as long per answer (65.2 against 69.6 seconds), and Ouro-2.6B-Thinking at prefill 4 / decode 2 scores higher on five of the eight benchmarks of Figure 4, by 3.5 to 3.8 points on MMLU-Pro, ARC-Challenge and BBH. On SenseNova-Loop-9B the trained prefill 1 / decode 1 scores 95.6 against 95.8 for native 2/2 on all 1,319 GSM8K problems and takes $0.59\times$ [0.52, 0.68] the time per answer, close to the ratio of layer executions per token (32 vs. 56).

4.5 Looped decision models: both directions

Untying is not specific to generation. Lev (Song et al., 2026) turns looped language models into decision models: they read a long context, then score the options of a question with a learned head after every loop, without generating text. We prefill the context with m loops, let each of the n decision loops attend to the last prefill loop's KV, and continue Lev's own LoRA and scoring head, once for every m; the control continues the same weights on the same data for the same number of updates with Lev's native n/n forward (Appendix J).

In these runs the sign of $m - n$ gives the direction of the difference (Figure 7, Table 15). With more prefill than decision loops the model is significantly above the control on 12 of 13 (m, n) pairs, most with one decision loop, where prefilling with all of the backbone's loops gains $+5.3$ [3.4, 7.2] on Lev-1.4B, $+7.5$ [5.7, 9.5] on Lev-2.6B and $+4.4$ [2.8, 6.1] on Lev-3B. With fewer prefill than decision loops it is below the control on all 13 pairs (significantly on 11), by up to 13.0 points (-13.0 [-14.7, -11.3]; Lev-2.6B, prefill 1, four decision loops). The fewer-prefill direction replicates over four training seeds: every seed is below the control on all 13 pairs, and on 10 of them significantly in every seed; the more-prefill comparison at the backbone's full loop count is replicated over four seeds and against a control trained twice as long (Appendix J). As in the task map, the context gains from many loops and the decision can then stop early.

5 Conclusion

A loop adapter makes the prefill and decode loops of an existing looped model independently configurable, reading aligned prompt caches or, to save memory, one stored cache. The two phases want different depths: with few decode loops, more prefill loops recover much of the accuracy of a deeper model, so a looped model can decode cheaply and prefill deeply, answering at native-4/4-level accuracy in $0.59\times$ the time. Looped decision models show the same rule from both sides. Untying the loops turns a fixed depth into two knobs, one paid once per prompt and one paid per token.

Limitations

The prefill count is chosen per benchmark on labeled items, not routed per prompt. The gain is largest at low decode depth; with three or four decode loops the aligned adapter matches but does not beat the native model, the single cache falls behind it on long-context retrieval, and a native model distilled on the same data closes part of the gap at two (Section 4.2). On few-shot MATH-500 one prefill loop is best (Appendix B). More prefill loops raise the time to first token; time is measured

at batch 1 on one GPU type. Thinking models are decoded greedily rather than sampled. All models are 1.4B–9B and benchmarks are English.

References

Kirill Afendulev, Alexey Dontsov, Elena Tutubalina, and Anton Korznikov. 2026. What Attention Recalls and Recurrence Controls in Hybrid Language Models. *arXiv preprint arXiv:2609.04434*.

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 117–134.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Abdulrahman Alabdulkareem, Irene Tenison, Yang Zhang, Yada Zhu, Boris Katz, and Dan Gutfreund. 2026. Swarm Telepathy: Prefill-Free KV-Cache Transfer Across Families. *arXiv preprint arXiv:2610.15095*.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. 2025a. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.20672.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025b. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*.

Andrea Banino, Jan Balaguer, and Charles Blundell. 2021. PonderNet: Learning to Ponder. *arXiv preprint arXiv:2107.05407*.

William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. 2024. Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2018. Universal Transformers. *arXiv preprint arXiv:1807.03819*.

Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, Zijia Chen, Ameya Sunil Mahabaleshwarkar, Shih-Yang Liu, Matthijs Van Keirsbilck, Min-Hung Chen, Yoshi Suhara, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. 2024. Hymba: A Hybrid-head Architecture for Small Language Models. *arXiv preprint arXiv:2411.13676*.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2020. Depth-Adaptive Transformer. In *International Conference on Learning Representations (ICLR)*.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. 2024. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Siqi Fan, Xuezhi Fang, Xingrun Xing, Peng Han, Shuo Shang, and Yequan Wang. 2025a. Position-Aware Depth Decay Decoding (D^3): Boosting Large Language Model Inference Efficiency. *arXiv preprint arXiv:2503.08524*.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. 2025b. Looped Transformers for Length Generalization. In *International Conference on Learning Representations (ICLR)*. ArXiv:2409.15647.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. 2025. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. *arXiv preprint arXiv:2510.03215*.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*.

Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. 2023. Looped Transformers as Programmable Computers. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 11398–11442.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. 2024. Think Before You Speak: Training Language Models With Pause Tokens. In *International Conference on Learning Representations (ICLR)*.

Alex Graves. 2016. Adaptive Computation Time for Recurrent Neural Networks. *arXiv preprint arXiv:1603.08983*.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2025. Training Large Language Models to Reason in a Continuous Latent Space. In *Conference on Language Modeling (COLM)*. ArXiv:2412.06769.

Junhui He, Zhihui Fu, Jun Wang, and Qingan Li. 2026. POP: Prefill-Only Pruning for Efficient Large Model Inference. *arXiv preprint arXiv:2602.03295*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring Massive Multitask Language Understanding. *arXiv preprint arXiv:2009.03300*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv preprint arXiv:2103.03874*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.

Zhiheng Hu, Yixun Wei, Jian Zhou, Yizhuang Zhou, Ji Li, Xing Chen, Yang Li, Bojun Wang, Yibo Zhu, Xiangyu Zhang, and Daxin Jiang. 2026. KITE: KV-Invariant Transformer Expansion for Efficient Agentic LLM Scaling. *arXiv preprint arXiv:2609.27294*.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. 2026. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*.

Woojeong Kim, Junxiong Wang, Jing Nathan Yan, Mohamed Abdelfattah, and Alexander M. Rush. 2025. OverFill: Two-Stage Models for Efficient Language Model Decoding. *arXiv preprint arXiv:2508.08446*.

Andreas Kirsch. 2026. LSTM-Style Looped Transformers with Shared KV Caches. *arXiv preprint arXiv:2610.33360*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

Faisal Ladhak, Shantanu Acharya, Santiago Akle Serano, Simeng Sun, Boris Ginsburg, and Krishna C Puvvada. 2026. One Model to Loop Them All: Elastic Latent Recurrence for Language Models. *arXiv preprint arXiv:2610.12582*.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *International Conference on Learning Representations (ICLR)*.

John Langford, Nathan Godey, Giovanni Monea, Yoav Artzi, Harry Dong, Ying Fan, Gustavo de Rosa, and Zheng Zhan. 2026. Almost Free State Prediction Separation. *arXiv preprint arXiv:2609.03807*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050*.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *arXiv preprint arXiv:2305.01210*.

Liming Liu, Mingze Wang, and Tuo Zhao. 2026. Decode-Branch Transformers: Decoupling the Primary Prefill Path from Additional Decode Computation. *arXiv preprint arXiv:2608.12385*.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. 2026. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*.

James O' Neill and Fergal Reid. 2026. Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers. *arXiv preprint arXiv:2607.15456*.

Jungsuk Oh, Hyeseo Jeon, Hyunjune Ji, Kyongmin Kong, and Jay-Yoon Lee. 2026. Shallow Prefill, Deep Decoding: Efficient Long-Context Inference via Layer-Asymmetric KV Visibility. *arXiv preprint arXiv:2605.06105*.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132.

Hayden Prairie, Jacob M. Ryan, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. 2026. BeNice! Closing Looping's Inference-Memory Gap via KV-Sharing. *arXiv preprint arXiv:2610.31633*.

Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, pages 155–170. ArXiv:2407.00079.

Haomiao Qiu, Yining Tang, Zijia Song, Ruizi Han, Ziyue Qiao, and Miao Zhang. 2026. CacheLink: Functional KV Transport for Zero-Prefill Cross-Model Context Handoff. *arXiv preprint arXiv:2610.15509*.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models. *arXiv preprint arXiv:2404.02258*.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *arXiv preprint arXiv:2311.12022*.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2020. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. 2025. Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *International Conference on Learning Representations (ICLR)*. ArXiv:2502.17416.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident Adaptive Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. 2026. Depth-adaptive Inference of Looped Language Models via Continuous Depth Batching. *arXiv preprint arXiv:2608.09444*.

Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150*.

Dingjie Song, Zhiling Yan, Yuxuan Zhang, and Lichao Sun. 2026. LEV: Thinking Fast and Deep with Looped Decision Models. *arXiv preprint arXiv:2610.20951*.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*.

Yutao Sun, Li Dong, Tianzhu Ye, Shaohan Huang, Jianyong Wang, and Furu Wei. 2026. Universal YOCO for Efficient Depth Scaling. *arXiv preprint arXiv:2604.01220*.

Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. 2024. You Only Cache Once: Decoder-Decoder Architectures for Language Models. *arXiv preprint arXiv:2405.05254*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv preprint arXiv:2210.09261*.

Victor Conchello Vendrell, Arnau Padres Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. 2026. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models. *arXiv preprint arXiv:2605.07721*.

Simon P. Villani. 2026. LatentPort: Beyond KV Cache - Cross-Model Transfer of Recurrent Memory in Hybrid Language Models: A 4B-to-9B Hybrid-State Handoff Without Target Prefix Replay. *arXiv preprint arXiv:2609.25053*.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *arXiv preprint arXiv:2406.01574*.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. 2025a. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*.

Bohong Wu, Shen Yan, Sijun Zhang, Jianqiao Lu, Yutao Zeng, Ya Wang, and Xun Zhou. 2025b. Efficient Pretraining Length Scaling. *arXiv preprint arXiv:2504.14992*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, and Bryan Dai. 2026. LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2606.18023*.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2024. Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464*.

Wanqi Yang and Shiwei Liu. 2026. FlashLoop: Fast and memory-efficient looped transformers via lazy updates. *arXiv preprint arXiv:2609.29812*.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2024. HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly. *arXiv preprint arXiv:2410.02694*.

Vincent-Daniel Yun, Woosang Lim, Haneul Yoo, Sungjoo Yoo, Murali Annavaram, and Sai Praneeth Karimireddy. 2026. Prefill-Free Cross-Family KV Cache Transfer for Heterogeneous Multi-Agent LLMs. *arXiv preprint arXiv:2609.32259*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.

Jiamu Zhang and Hanjie Chen. 2026. BellowsKV: One Latent for Many Layers, Expanded on Demand. *arXiv preprint arXiv:2610.35023*.

Yudong Zhang, Wei Ding, Kaili Zheng, Yuchen Ding, and Zhenyu Zhang. 2026. Accordion: Elastic-Depth Inference via Fixed-Prefix Layer Looping. *arXiv preprint arXiv:2610.32492*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. *arXiv preprint arXiv:2311.07911*.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, and 14 others. 2025. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*.

Zayd Muhammad Kawakibi Zuhri, Muhammad Farid Adilazuarda, Ayu Purwarianti, and Alham Fikri Aji. 2025. MLKV: Multi-Layer Key-Value Heads for Memory Efficient Transformer Decoding. In *Findings of the Association for Computational Linguistics: NAACL 2025*.

A Training Details

Loop adapter. Rank-16 LoRA ($\alpha = 16$, no dropout) on the query, key, value and output projections, active only when decoding; per (decode loop, layer, KV head) a complex scale for each rotary frequency pair on keys and a full $D \times D$ value map, both initialised to the identity. Trainable parameters: 63,307,776 for Ouro-2.6B. The backbone is frozen.

Objective and data. KL(teacher ‖ student) summed over answer positions and normalised by $\sum_i(|a_i| - 1)$ over the answers a_i of the global batch (one less than the supervised positions per answer, as the released checkpoints were trained); teacher = native 4/4 with the same weights; one pair (m, n) sampled per micro-batch. 48M answer tokens: OpenThoughts reasoning traces 45%, Tulu SFT responses 40%, LongAlpaca long-context answers 5% and FineWeb-Edu windows 10% (fractions of answer tokens); prompts up to 7,680 tokens, answers up to 1,024; the token budget counts $|a| - 1$ positions per answer a for the loop adapter and the distilled controls alike. Base models see a plain "User: … Assistant:" format; thinking models see the same samples in their chat template. No benchmark data was deliberately included; we did not decontaminate the training corpora. An audit afterwards (`overlap_audit.py`) finds a 13-token span shared with the training data in 187 of 542 code prompts (HumanEval+ and MBPP+), 101 of 500 MATH-500 questions, 106 of 14,042 MMLU, 75 of 6,511 BBH, 24 of 1,400 MMLU-Pro and at most 3 items of each other audited benchmark (GSM8K, ARC-Challenge, HellaSwag, Winogrande, GPQA, IFEval). Items whose extracted question text is unavailable or shorter than 13 tokens cannot be assessed (2,696 MMLU, 296 HellaSwag, 254 BBH, 247 ARC-Challenge, 194 MMLU-Pro, at most 4 elsewhere) and are kept; RULER and HELMET are synthetic or long-document and not audited. Without the overlapping items the chosen m of Table 5 still wins significantly on 13 of 15 and 9 of 15 settings at one and two decode loops; at one decode loop MBPP+, which loses 93 of its 378 problems, becomes inconclusive ($+2.5$ $[-1.1, 10.2]$).

Optimisation. AdamW ($\beta = (0.9, 0.95)$, no weight decay), peak learning rate 10^{-4} with 2% linear warm-up and cosine decay to 10^{-5}, gradient clipping at 1.0, global batch 24 sequences on 4 B300 GPUs (6.75 hours for Ouro-2.6B), bfloat16 backbone.

Shared answer KV cache. Generated tokens keep one KV cache per layer shared by all decode loops; at inference the last decode loop commits each token, and earlier loops read that history plus the current token's KV computed live in their own loop (*self-live*[1], a layout also used by LSTM-style looped transformers (Kirsch, 2026)). For example, when loop 1 processes token 5, it attends to tokens 1–4 through the last loop's stored KV and to token 5 through its own, freshly computed KV. Training uses the same KL objective in this layout, in two passes: a no-grad pass in which each loop reads its own causal answer KV records the last loop's answer KV; a second pass reads, at every loop, the prompt KV cache, that recorded history strictly before the current position, and the current token's KV computed live on the diagonal. Training thus approximates, but does not exactly reproduce, inference, where the history is generated recursively in the shared cache. With prompt and answer each stored once, total KV is $b(\ell + t)$, that of a one-loop model; with 1K-token prompts and 2,048 generated tokens the largest batch that fits grows from 32 (native 4/4) to 192 and decode-4 throughput from 219 to 476 tokens/s (759 vs. 586 at n=2), at a cost of 3.0 GSM8K points at 4/4 against per-loop answer caches (Appendix I, Other design variants). On Ouro-2.6B-Thinking, a loop adapter trained the same way changes full-set GSM8K at prefill 4 / decode 2 by $+0.5$ $[-0.5, 1.6]$ against per-loop answer caches; at batch 1 its observed cost per generated token is similar (82.5 vs. 83.1 ms) but its answers are longer (1,166 vs. 741 tokens), so it takes $1.53\times$ $[1.27, 1.84]$ the time per answer. Its benefit is memory and batch size, not batch-1 latency.

SenseNova-Loop-9B. Rank-16 LoRA on the attention projections of the looped layers (full attention: q/k/v/o; Gated-DeltaNet: input and output projections), active only when decoding; no maps. The one-visit decoder reads the prefill state of the last of $m \in \{1, 2\}$ visits (sampled 50/50), so the same decoder serves prefill 1 and prefill 2. The untrained condition passes the last visit's full-attention KV and Gated-DeltaNet convolution and recurrent states. Teacher: native 2/2; 20M answer tokens of the same mixture in the SenseNova chat template; peak learning rate 2×10^{-4}.

B Evaluation Details

Protocol. Base models follow the base-model protocol of the Ouro models (Zhu et al., 2025, Appendix C.1), run through our runner so that

[1] Self-live was developed by the author in separate work on shared KV caches for looped models; here it is applied to the loop adapter's answer tokens.

every (m, n) pair is evaluated the same way: lm-evaluation-harness for MMLU (5-shot, log-likelihood, 14,042), MMLU-Pro (5-shot CoT, strict match; 30 per subject, 420), BBH (3-shot CoT, strict match; 60 per subtask, 1,620), ARC-Challenge (25-shot, 1,172), HellaSwag (10-shot, 10,042), Winogrande (5-shot, 1,267) and GSM8K (3-shot CoT, strict match, 1,319); MATH-500 with the 4-shot Minerva prompt, scored on the final answer (the last `\boxed{}`, else the Minerva closing sentence) with math-verify (the Ouro models' own 5-shot prompt and scorer are not released); HumanEval+ and MBPP+ with evalplus as it runs for a tokenizer with a chat template (the problem in evalplus's chat prompt, greedy, 768 new tokens, sanitized code, plus tests). With this protocol native Ouro-2.6B 4/4 scores within one point of the published numbers on MMLU, MMLU-Pro, BBH, ARC-Challenge and HellaSwag and within 1.2 to 3.1 points on HumanEval(+) and MBPP(+) (Table 3). Two scores differ for a documented reason: GSM8K strict matching requires the exemplars' "The answer is" sentence and gives 69.4, while flexible extraction of the last number gives 86.7, bracketing the published 81.6; and MATH-500 with the public 4-shot Minerva prompt gives 53.6 against 90.9 with Ouro's unreleased 5-shot prompt and scorer. Every configuration in this paper is scored the same way, so these choices shift all rows together.

benchmark	ours	reported	items
MMLU	74.5	74.6	14,042
MMLU-Pro	56.1	55.7	1,400
BBH	80.7	80.5	6,511
ARC-C	66.5	66.4	1,172
HellaSwag	79.5	79.7	10,042
Winogrande	77.7	75.8	1,267
GSM8K	69.4	81.6	1,319
GSM8K (flexible extraction)	86.7	–	1,319
MATH-500	53.6	90.8	500
HumanEval	75.6	78.7	164
HumanEval+	69.5	70.7	164
MBPP	77.5	80.4	378
MBPP+	64.5	66.6	378

Table 3: **Reproducing the native model.** Ouro-2.6B 4/4 under our evaluation and as reported by Zhu et al. (2025, Table 8); MMLU-Pro and BBH here on the full runs of the native model.

GPQA-Diamond (0-shot), IFEval (prompt-level strict), RULER (13 tasks $\times$ 40 rows per length) and HELMET (RAG, synthetic recall, re-ranking and ICL; 1,108 rows) are prepared and scored by our own scripts. Thinking models answer GSM8K and MATH-500 zero-shot in their chat template, answers in `\boxed{}` scored with math-verify, with budgets of 4,096 and 8,192 tokens, greedily rather than with the sampling and model-judged scoring of their original evaluation; their MMLU-Pro and BBH are as for base models and their code as above. SenseNova-Loop-9B uses MMLU-Pro with 70 questions per subject (980) and the full BBH (6,511) through our own scripts. On raw HELMET prompts thinking checkpoints can emit a newline and stop; for thinking models we therefore mask the stop tokens until an answer has a non-whitespace token, for every configuration. Intervals: paired bootstrap over items, 5,000 resamples, recomputed from released per-item scores by one script (Appendix M).

Scoring and prompts. lm-evaluation-harness's strict Minerva match for MATH-500 credits an answer only if it ends with the exemplars' sentence "Final Answer: The final answer is X. I hope it is correct." Native Ouro-1.4B 4/4 writes boxed answers that are mostly correct but closes them in other words, so it scores 4.4 under strict matching and 70.6 on the final answer; on Ouro-2.6B the decoder with one prefill loop and four decode loops writes longer solutions than the exemplars, ends with that sentence on 183 of 500 problems (429 with four prefill loops), and scores 18.6 under strict matching but 68.2 on the final answer (51.0 with four prefill loops). We therefore score the final answer; the prefill-1 advantage on 4-shot MATH-500 is in the answers, not in their format. Code is different: under an earlier lm-evaluation-harness prompt, prefilling base Ouro-2.6B with two or more loops dropped Python indentation in 35–63% of MBPP+ answers (under 3% with one), and that task scored only the first base assertion; with evalplus this effect is gone. Answer format also changes one comparison of Appendix F: for Nanbeige4.2-3B-Base, prefill 2 / decode 1 minus prefill 1 / decode 1 on GSM8K is -8.4 with strict matching but positive with flexible extraction. Native throughput baselines: static per-loop caches with the same two-call flash attention (fastest), the standard dynamic cache (500 / 245 / 105 tokens/s at $n = 1/2/4$), and a preallocated contiguous cache with one attention call (267 / 130 / 57); all produce identical greedy tokens.

C Two-Seed Checks on Selected Comparisons

	task	seed 1	seed 2
prefill 4 − prefill 1 (decode 1)	GSM8K	$+8.5$	$+8.9$
	MMLU	$+15.2$	$+15.2$
	RULER-4K	-3.8	-3.5
	RULER-8K	$+2.5$	$+2.0$
	RULER-16K	$+8.6$	$+8.6$
prefill 4 − 2 (decode 2)	GSM8K	-0.1	-1.1
	RULER-4K	-7.2	-6.8

Table 4: Ouro-2.6B loop adapter trained with two seeds (initialisation and (m, n) sampling); paired differences within each seed, full sets.

D Choosing the Prefill Loops per Benchmark

benchmark	decode 1	decode 2	decode 3	decode 4
MMLU	$+15.1^{*}$ [14.2, 16.0] (3,4)	$+5.6^{*}$ [4.8, 6.3] (4)	$+0.6$ [−0.2, 1.1] (4)	-0.5^{*} [−1.1, −0.2] (4)
GSM8K	$+13.2^{*}$ [9.2, 15.7] (2)	$+5.7^{*}$ [1.9, 7.7] (3)	$+0.5$ [−1.9, 2.3] (3)	$+1.0$ [−1.7, 2.7] (3)
MATH-500 4-shot	$+5.6^{*}$ [1.8, 9.2] (1)	$+14.4^{*}$ [10.2, 18.4] (1)	$+14.2^{*}$ [10.0, 18.4] (1)	$+14.6^{*}$ [10.4, 18.8] (1)
BBH 3-shot	$+20.9^{*}$ [17.1, 23.6] (4)	$+6.5^{*}$ [3.0, 8.8] (3)	-2.3 [−3.9, 0.4] (4,2)	-1.7^{*} [−4.3, −0.2] (3)
ARC-C	$+14.8^{*}$ [10.6, 17.2] (4)	$+2.7^{*}$ [0.5, 4.4] (4,3)	$+0.9$ [−1.0, 1.9] (3)	-0.2 [−2.5, 0.7] (3)
MMLU-Pro	$+19.0^{*}$ [13.6, 25.2] (4,3)	$+5.7$ [−0.5, 10.5] (4,3)	$+0.7$ [−5.2, 5.0] (4)	-2.1 [−8.3, 2.6] (4,3)
HellaSwag	$+3.9^{*}$ [3.1, 4.6] (3)	$+0.5$ [−0.2, 0.9] (3)	$+0.0$ [−0.4, 0.4] (3,4)	$+0.0$ [−0.5, 0.4] (3,4)
Winogrande	$+6.2^{*}$ [2.2, 9.2] (3)	$+2.2^{*}$ [0.1, 5.1] (3,4)	$+0.1$ [−2.7, 1.7] (2)	-0.9 [−3.2, 0.6] (2)
HumanEval+	$+21.3^{*}$ [11.6, 28.7] (3)	$+7.9$ [0.0, 12.8] (3)	$+1.2$ [−6.7, 5.5] (3)	$+3.7$ [−3.0, 7.9] (3)
MBPP+	$+4.2^{*}$ [0.8, 9.8] (4,2)	$+2.4$ [−1.9, 5.0] (2)	-0.5 [−3.7, 2.1] (2,3)	-0.5 [−6.1, 2.4] (2)
GPQA-Diamond	$+12.1$ [−2.5, 18.7] (4)	$+3.5$ [−7.1, 11.6] (3,4)	-1.0 [−9.1, 7.6] (4,2)	-3.5 [−10.6, 4.5] (3,4)
IFEval	$+9.4^{*}$ [4.1, 12.8] (4)	-0.9 [−5.5, 3.5] (4,3)	$+1.8$ [−2.8, 4.8] (2)	$+0.4$ [−4.3, 4.3] (4,2)
RULER-4K	$+12.3^{*}$ [6.2, 15.9] (2)	$+5.7^{*}$ [0.3, 7.9] (2)	-1.6^{*} [−6.1, −0.01] (2)	-2.3^{*} [−6.3, −0.6] (2)
RULER-8K	$+17.6^{*}$ [10.4, 21.6] (3)	$+5.6^{*}$ [2.4, 10.6] (2,3)	-9.0^{*} [−11.8, −5.5] (2,3)	-10.3^{*} [−13.2, −6.0] (2,3)
RULER-16K	$+19.6^{*}$ [15.9, 25.2] (4,3)	$+10.4^{*}$ [5.8, 15.4] (4,3)	-5.6^{*} [−9.8, −2.0] (3,4)	-5.2^{*} [−9.8, −2.6] (3)

Table 5: **Choosing the prefill loops per benchmark, against the native model with the same decode loops.** Ouro-2.6B, one loop adapter. For each benchmark and number of decode loops n we choose the prefill count m on one half of the items, score it on the other half, and swap, so that no item is scored by a choice made on it; the table gives the resulting accuracy minus native n/n on the same items, the m chosen on each half (in parentheses), and paired 95% bootstrap intervals that repeat the choice of m in every resample (*: interval excludes zero; nominal, per benchmark). Score points on a 0–100 scale.

E Full Task Map

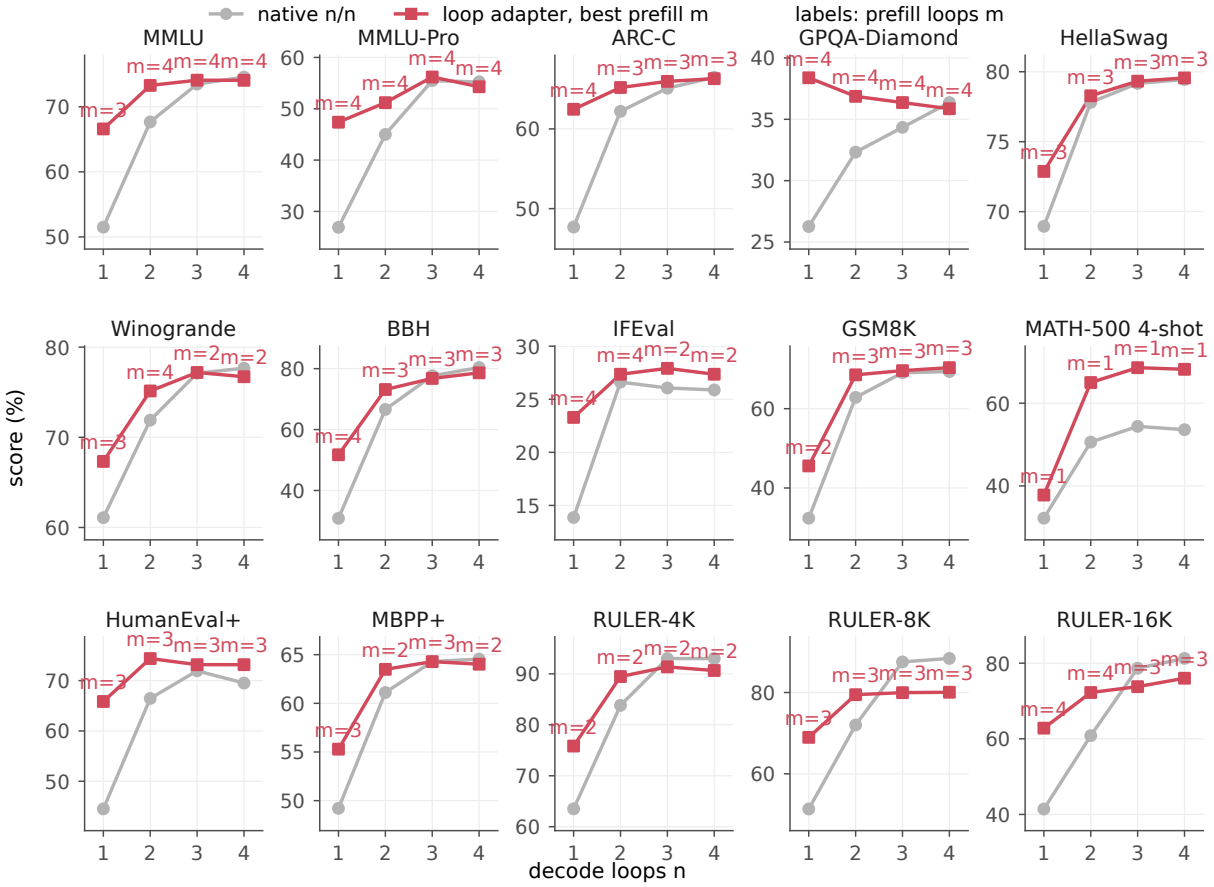


Figure 8: **All 15 task-map settings.** Ouro-2.6B, one loop adapter; grey: native n/n; red: the best prefill count m at each n (labels; the smallest maximizing m), chosen on these items; the held-out choice is Table 5. Every cell: Table 6.

	MMLU	GSM8K	MATH	BBH	ARC-C	MMLU-P	HSwag	Wino	HE+	MBPP+	GPQA	IFEv	R-4K	R-8K	R-16K
native 1/1	51.5	32.4	32.2	30.9	47.7	26.9	69.0	61.1	44.5	49.2	26.3	13.9	63.5	51.3	41.4
1/1	51.4	36.2	**37.8**	37.9	52.6	27.9	69.2	65.8	51.2	48.4	26.3	21.6	74.1	64.3	54.2
2/1	63.5	**45.6**	25.6	50.3	59.7	43.1	72.5	66.3	59.1	54.2	30.3	22.0	**75.8**	66.9	61.6
3/1	**66.6**	43.3	25.4	51.4	61.5	46.2	**72.9**	**67.3**	**65.9**	**55.3**	32.8	21.4	74.0	**69.0**	61.3
4/1	66.6	44.7	25.8	**51.7**	**62.5**	**47.4**	72.4	67.1	64.6	55.3	**38.4**	**23.3**	70.2	66.8	**62.8**
native 2/2	67.7	62.9	50.6	66.6	62.2	45.0	77.8	71.9	66.5	61.1	32.3	26.6	83.8	72.0	60.8
1/2	53.3	56.6	**65.0**	56.5	58.2	33.1	74.7	68.3	64.6	56.9	26.8	22.7	77.9	69.7	57.6
2/2	67.3	66.8	49.6	71.4	64.1	47.4	77.8	74.2	64.6	**63.5**	34.3	25.7	**89.5**	78.3	66.9
3/2	72.9	**68.5**	48.4	**73.1**	**65.2**	51.0	**78.3**	74.4	**74.4**	63.0	36.4	26.6	85.5	**79.5**	71.6
4/2	**73.3**	66.7	47.0	71.2	65.1	**51.2**	78.0	**75.1**	68.3	62.7	**36.9**	**27.4**	82.3	77.9	**72.2**
native 3/3	73.5	69.1	54.4	77.7	65.1	55.5	79.2	77.1	72.0	64.3	34.3	26.1	93.0	87.5	78.7
1/3	53.0	60.5	**68.6**	64.0	58.5	32.6	76.0	68.5	66.5	59.3	27.3	22.4	80.3	71.5	58.1
2/3	67.4	68.8	53.4	76.3	64.8	49.3	79.1	**77.2**	70.1	63.8	34.3	**27.9**	**91.4**	79.2	69.3
3/3	73.5	**69.6**	53.2	**76.8**	**66.0**	53.3	**79.3**	75.5	**73.2**	**64.3**	34.3	26.8	87.2	**80.0**	**73.8**
4/3	**74.1**	67.6	52.2	76.5	65.4	**56.2**	79.2	76.1	68.3	62.2	**36.4**	25.7	82.8	77.9	73.4
native 4/4	74.6	69.4	53.6	80.4	66.5	55.2	79.4	77.7	69.5	64.5	36.4	25.9	93.0	88.4	81.3
1/4	52.9	60.7	**68.2**	64.5	57.9	33.1	76.2	69.1	69.5	59.0	29.8	24.6	80.4	70.1	57.0
2/4	67.5	69.0	54.8	77.0	65.2	47.9	79.1	**76.7**	67.7	**64.0**	34.3	**27.4**	**90.7**	78.2	68.3
3/4	73.5	**70.4**	54.8	**78.6**	**66.3**	53.6	**79.6**	75.5	**73.2**	63.8	32.8	26.8	87.6	**80.1**	**76.0**
4/4	**74.1**	68.8	51.0	78.0	65.3	**54.3**	79.5	75.7	66.5	62.7	**35.9**	27.2	82.3	76.6	73.4

Table 6: **Every cell of the task map.** Ouro-2.6B, accuracy (%) of native n/n and of one loop adapter at prefill m / decode n (m / n), each column on the items that all 20 configurations share; bold: the best m at each n, chosen on these items (Figures 1 and 8; the held-out choice is Table 5). MMLU and GSM8K are the full sets (14,042 and 1,319 items); RULER uses 520 items per length. MMLU-P: MMLU-Pro, HEval: HumanEval, IFEv: IFEval, R-kK: RULER at kK tokens.

benchmark	decode 1	decode 2	decode 3
MMLU	$+13.8^{*}$ [12.9, 14.8]	$+6.1^{*}$ [5.3, 6.8]	$+1.0^{*}$ [0.3, 1.5]
GSM8K	$+5.2^{*}$ [1.3, 7.6]	-1.1 [-4.8, 0.9]	-1.7 [-4.2, 0.0]
MATH-500 4-shot	$+0.8$ [-3.0, 4.4]	$+18.2^{*}$ [14.0, 22.4]	$+14.8^{*}$ [10.6, 19.0]
BBH 3-shot	$+11.3^{*}$ [7.4, 14.2]	$+2.7$ [-0.7, 4.6]	-2.3 [-3.8, 0.4]
ARC-C	$+7.5^{*}$ [3.7, 9.6]	$+2.0$ [-0.1, 3.7]	$+0.5$ [-1.3, 1.5]
MMLU-Pro	$+19.3^{*}$ [14.0, 25.5]	$+4.5$ [-1.9, 9.3]	$+2.9$ [-2.6, 6.9]
HellaSwag	$+2.7^{*}$ [2.1, 3.2]	$+0.7^{*}$ [0.1, 1.1]	-0.0 [-0.4, 0.4]
Winogrande	$+0.9$ [-2.8, 3.4]	-0.5 [-2.4, 2.2]	$+0.1$ [-2.6, 1.7]
HumanEval+	$+11.6^{*}$ [2.4, 18.3]	$+4.9$ [-2.4, 9.8]	$+4.3$ [-3.7, 9.1]
MBPP+	$+3.2$ [-0.8, 9.0]	$+0.5$ [-3.7, 3.2]	-0.3 [-3.7, 2.7]
GPQA-Diamond	$+10.6$ [-3.5, 17.2]	$+4.5$ [-5.6, 11.1]	$+1.0$ [-5.6, 9.1]
IFEval	$+2.2$ [-3.0, 5.2]	$+0.7$ [-4.1, 5.0]	$+1.5$ [-3.1, 4.4]
RULER-4K	-3.5^{*} [-9.4, -0.4]	-1.4^{*} [-6.5, -0.1]	-1.8^{*} [-6.3, -0.2]
RULER-8K	$+0.8$ [-6.2, 4.1]	-5.4^{*} [-8.2, -0.7]	-8.7^{*} [-11.5, -5.3]
RULER-16K	$+6.3^{*}$ [3.1, 11.6]	$+0.9$ [-3.3, 5.5]	-8.0^{*} [-12.1, -4.5]

Table 7: **At one decode loop, choosing the prefill loops beats distilled native models on knowledge, code and few-shot math.** Ouro-2.6B: the m chosen as in Table 5, minus native n/n distilled with LoRA (same data, teacher and 48M tokens as the loop adapter, supervising every answer token as the adapter does), on the same items; selection-aware paired 95% intervals (*: excludes zero). At two decode loops the counts are mixed (Section 4.2); at four decode loops the distilled model is the teacher itself.

F Other Checkpoints

Table 9 compares prefill counts at one decode loop within the same loop adapter on five checkpoints from two model families.

Table 8 repeats the per-benchmark choice on Ouro-1.4B. As on Ouro-2.6B, the gain is largest with one decode loop (significant on 12 of 15 settings) and shrinks with more (2 of 15 at two decode loops, where 3-shot BBH is significantly worse than native 2/2); within the adapter the chosen m beats the adapter's own n/n significantly on 6 and 1 of 15. Against native 1/1 and 2/2 distilled with LoRA on the same data, teacher and tokens, it is significantly above on 6 and below on 3 of 15 at one decode loop (below on all three RULER lengths, where distillation alone helps more), and above on 4 and below on 4 at two. Nanbeige4.2-3B-Base, a non-looped model run with its blocks repeated and one loop adapter trained per (m, n) pair, shows the same direction at one decode loop: prefill 2 / decode 1 minus prefill 1 / decode 1 is $+11.8$, $+11.7$ and $+14.2$ on RULER 4K/8K/16K and $+12.2$ on MMLU (GSM8K: Appendix B).

benchmark	decode 1	decode 2	decode 3	decode 4
MMLU	$+23.4^{*}$ [19.2, 26.7] (3)	$+5.0^{*}$ [2.5, 7.8] (4,3)	$+0.3$ [-1.4, 2.6] (3,4)	-0.2 [-1.5, 1.5] (3,4)
GSM8K	$+9.0^{*}$ [4.8, 14.8] (2,4)	$+6.2$ [0.0, 10.8] (3)	$+0.2$ [-6.2, 4.2] (3)	$+2.8$ [-5.0, 6.5] (3)
MATH-500 4-shot	$+2.6$ [-2.2, 7.0] (3,1)	$+0.2$ [-7.2, 3.8] (3)	-6.4^{*} [-9.8, -1.4] (3,2)	-2.0 [-8.4, 1.2] (1)
BBH 3-shot	$+12.8^{*}$ [10.6, 15.1] (1)	-5.4^{*} [-10.7, -2.9] (1)	-4.0^{*} [-7.8, -1.9] (3)	-2.9^{*} [-6.5, -1.0] (3)
ARC-C	$+17.2^{*}$ [13.8, 20.1] (4)	$+4.2^{*}$ [2.0, 6.5] (3,4)	$+1.1$ [-0.9, 2.6] (4)	-0.1 [-2.0, 1.1] (4)
MMLU-Pro	$+17.9^{*}$ [10.7, 23.1] (3)	$+6.2$ [-1.2, 10.5] (3)	$+2.9$ [-2.6, 6.7] (3)	$+0.5$ [-3.8, 4.8] (4,3)
HellaSwag	$+8.0^{*}$ [7.0, 8.9] (3)	$+0.3$ [-0.4, 0.9] (3)	-0.2 [-0.7, 0.2] (3)	-0.6^{*} [-1.1, -0.2] (3)
Winogrande	$+6.8^{*}$ [2.5, 10.3] (3,2)	$+2.3$ [-1.0, 4.7] (2)	-2.8^{*} [-5.1, -0.4] (3,2)	-1.1 [-4.1, 0.5] (3)
HumanEval+	$+5.5$ [0.0, 14.0] (2,4)	$+5.5$ [-2.4, 13.4] (4,3)	$+0.6$ [-4.9, 9.1] (4,1)	-3.7 [-7.9, 4.3] (4,1)
MBPP+	$+9.0^{*}$ [1.9, 13.2] (3)	$+0.5$ [-5.3, 4.2] (3)	$+2.4$ [-3.4, 5.0] (3)	$+2.1$ [-2.1, 4.5] (3)
GPQA-Diamond	-3.5 [-11.6, 11.1] (3,1)	$+6.1$ [-2.5, 15.2] (3,4)	-7.1 [-14.6, 0.5] (3,4)	-3.5 [-13.1, 0.5] (4)
IFEval	$+7.2^{*}$ [3.0, 12.4] (2,4)	-1.1 [-5.0, 3.1] (2,3)	$+0.4$ [-6.5, 3.7] (2)	-3.0 [-6.3, 1.3] (2,3)
RULER-4K	$+14.5^{*}$ [11.0, 19.3] (1,3)	$+4.1$ [-2.7, 7.7] (1)	-11.7^{*} [-20.0, -8.4] (1)	-13.2^{*} [-22.2, -9.8] (1)
RULER-8K	$+14.5^{*}$ [9.2, 18.2] (2)	$+0.9$ [-2.8, 5.5] (2,1)	-12.0^{*} [-17.7, -8.7] (1)	-17.8^{*} [-21.4, -11.6] (3,1)
RULER-16K	$+13.4^{*}$ [9.9, 17.6] (2,4)	$+1.5$ [-3.3, 5.6] (4)	-6.2^{*} [-10.4, -1.4] (1,4)	-18.4^{*} [-22.1, -13.0] (1,3)

Table 8: **Table 5 on Ouro-1.4B** (its own loop adapter; the 15 settings of Table 5; MMLU and GSM8K subsets of 20 questions per subject and 400 problems): prefill loops chosen on one half of the items, scored on the other, minus native n/n on the same items, in points; the m chosen on each half in parentheses; paired 95% bootstrap intervals that repeat the choice of m in every resample (*: excludes zero).

model (same decoder, decode 1)	knowledge	long context	multi-step
Ouro-2.6B, prefill 4 vs. 1	MMLU $+15.2$ [14.3, 16.1]	RULER-16K $+8.6$ [4.3, 12.8]	GSM8K $+8.5$ [6.1, 10.9]
Ouro-1.4B, prefill 4 vs. 1	ARC-C $+10.8$ [8.4, 13.1]	RULER-16K $+1.4$ [-2.6, 5.4]	GSM8K $+5.0$ [2.7, 7.2]
Ouro-2.6B-Thinking, prefill 4 vs. 1	MMLU-Pro $+18.6$ [13.6, 23.8]	HELMET $+7.7$ [2.1, 13.6]	GSM8K $+6.6$ [4.2, 9.2]
Ouro-1.4B-Thinking, prefill 4 vs. 1	MMLU-Pro $+17.1$ [12.4, 22.4]	HELMET $+4.6$ [0.1, 8.6]	GSM8K $+13.6$ [10.5, 16.6]
SenseNova-Loop-9B, prefill 2 vs. 1	MMLU-Pro $+5.5$ [2.9, 8.1]	HELMET $+3.5$ [0.8, 6.1]	BBH $+7.0$ [5.9, 8.0]

Table 9: **Prefill loops with the decoder held fixed** across five checkpoints (paired, within one loop adapter). Set sizes: MMLU 14,042, ARC-C 1,172, MMLU-Pro 420 (Ouro) and 980 (SenseNova), RULER 520, HELMET 1,108 (Ouro) and 6,646 (SenseNova), GSM8K 1,319, BBH 6,511. HELMET is averaged as its authors do (mean per dataset, then per category, then over its four categories). Each column shows one benchmark of that kind per checkpoint; counterexamples are in Appendix L. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

G Prefill 4 / Decode 2 in Numbers, and on Ouro-1.4B-Thinking

Table 10 lists the scores and paired intervals behind Figure 4 and repeats the comparison on the smaller Thinking checkpoint. The effect of prefill loops replicates: with the same loop adapter, prefilling with four loops instead of two raises MMLU-Pro ($+8.6$), HELMET ($+7.3$), ARC-Challenge ($+2.7$), BBH ($+8.8$), HumanEval+ ($+7.9$) and MBPP+ ($+6.3$), with no detectable difference on GSM8K and MATH-500. Against native 2/2 the result is weaker than on Ouro-2.6B-Thinking: prefill 4 / decode 2 is significantly better on GSM8K ($+3.0$), ARC-Challenge ($+4.3$) and HumanEval+ ($+10.4$), within noise on four benchmarks, and much worse on 3-shot BBH (-15.9): on this raw few-shot prompt the adapter often ends its answer right after the exemplars' opening "Let's think step by step." (734 of 1,620 answers at prefill 4 / decode 2, 28 for native 2/2), an early stop that its chat-format training data never show; Ouro-2.6B-Thinking does not do this. Against native 4/4 the gap is larger than on Ouro-2.6B-Thinking, most of all on BBH (-37.2) and HELMET (-18.3).

H End-to-End Time and Decode Throughput

Serving. Storing the prompt once also changes throughput (Figure 5; Table 13 in Appendix H). In a controlled decode benchmark (8K context, 32 repeated decode steps, largest batch that fits on one B300), one prompt KV cache decodes $0.97\times$ as fast as the fastest native implementation we measured at one decode loop and 1.19–$1.62\times$ at two to four, because it admits larger batches as n grows; this native baseline uses the same flash attention and is faster than the standard dynamic cache (Appendix B). With long answers (1K-token prompts, 2,048 generated tokens) one prompt KV cache decodes $0.97\times$ native at $n{=}2$ and $1.16\times$ at $n{=}4$.

	score				adapter 4/2 minus		
benchmark	native 2/2	adapter 2/2	adapter 4/2	native 4/4	native 2/2	adapter 2/2	native 4/4
Ouro-2.6B-Thinking (Figure 4)							
GSM8K	92.2	93.0	93.3	94.2	+1.1 [−0.2, 2.4]	+0.2 [−1.0, 1.4]	−1.0 [−2.0, 0.1]
MATH-500	84.8	84.8	85.0	89.8	+0.2 [−2.6, 3.0]	+0.2 [−2.8, 3.2]	−4.8 [−7.8, −2.0]
MMLU-Pro	43.1	41.9	50.0	49.0	+6.9 [3.1, 11.0]	+8.1 [4.0, 12.1]	+1.0 [−2.6, 4.5]
ARC-C	62.5	63.8	66.1	67.5	+3.6 [2.0, 5.3]	+2.3 [0.9, 3.7]	−1.4 [−2.6, −0.1]
BBH	66.6	71.3	73.5	71.7	+6.9 [4.6, 9.2]	+2.2 [0.1, 4.3]	+1.9 [−0.4, 4.1]
HumanEval+	68.9	70.7	74.4	73.8	+5.5 [−0.6, 11.6]	+3.7 [−2.4, 9.8]	+0.6 [−4.9, 6.1]
MBPP+	61.9	61.6	62.4	60.6	+0.5 [−2.9, 4.0]	+0.8 [−2.4, 4.0]	+1.9 [−0.5, 4.2]
HELMET	44.0	49.5	57.0	65.7	+13.0 [6.7, 19.5]	+7.5 [1.4, 13.8]	−8.7 [−13.3, −4.1]
Ouro-1.4B-Thinking							
GSM8K	86.4	89.5	89.5	92.3	+3.0 [1.3, 4.8]	+0.0 [−1.5, 1.4]	−2.9 [−4.3, −1.5]
MATH-500	76.4	77.6	75.6	85.2	−0.8 [−4.6, 3.0]	−2.0 [−5.8, 1.8]	−9.6 [−12.8, −6.2]
MMLU-Pro	39.8	34.8	43.3	46.2	+3.6 [−1.0, 8.1]	+8.6 [4.3, 12.9]	−2.9 [−6.7, 1.2]
ARC-C	56.1	57.7	60.4	62.5	+4.3 [2.1, 6.4]	+2.7 [1.0, 4.5]	−2.0 [−3.7, −0.4]
BBH	48.7	24.0	32.8	70.0	−15.9 [−18.9, −13.0]	+8.8 [6.2, 11.5]	−37.2 [−39.9, −34.5]
HumanEval+	59.1	61.6	69.5	73.2	+10.4 [4.3, 17.1]	+7.9 [1.8, 14.0]	−3.7 [−8.5, 1.2]
MBPP+	55.3	52.9	59.3	62.4	+4.0 [−0.3, 8.2]	+6.3 [2.4, 10.3]	−3.2 [−6.1, −0.3]
HELMET	45.0	39.2	46.5	64.9	+1.5 [−4.8, 7.8]	+7.3 [2.2, 12.7]	−18.3 [−22.8, −13.6]

Table 10: **Figure 4 in numbers, and on Ouro-1.4B-Thinking.** Per checkpoint, one loop adapter serves prefill 2 / decode 2 and prefill 4 / decode 2; same benchmarks, protocols and items for both checkpoints. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

	score			distilled 2/2	prefill 4 / decode 2
benchmark	native 2/2	distilled 2/2	prefill 4 / decode 2	minus native 2/2	minus distilled 2/2
GSM8K	92.2	93.0	93.3	+0.8 [−0.4, 2.0]	+0.2 [−1.0, 1.4]
MATH-500	84.2	83.2	83.8	−1.0 [−4.5, 2.5]	+0.5 [−2.5, 3.2]
MMLU-Pro	43.1	44.0	50.0	+1.0 [−2.9, 4.5]	+6.0 [1.7, 10.2]
ARC-C	62.5	63.0	66.1	+0.4 [−0.7, 1.5]	+3.2 [1.5, 4.8]
BBH	66.6	71.4	73.5	+4.8 [2.8, 6.9]	+2.1 [−0.1, 4.3]
HumanEval+	68.9	69.5	74.4	+0.6 [−4.3, 5.5]	+4.9 [−0.6, 11.0]
MBPP+	61.9	61.6	62.4	−0.3 [−2.6, 2.1]	+0.8 [−2.6, 4.5]
HELMET	44.0	53.8	57.0	+9.8 [6.9, 12.9]	+3.2 [−2.7, 9.3]

Table 11: **Untying versus distillation** (Ouro-2.6B-Thinking). "Distilled 2/2": rank-16 LoRA on the native two-loop model, trained with the same data, teacher (native 4/4) and number of tokens as the loop adapter. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

task / model	config	acc. (%)	s / answer	tokens	N
GSM8K Ouro-2.6B-T	native 2/2	92.7	63.3	991	192
	native 4/4	92.7	118	878	192
	distilled 2/2	94.3	57.7	837	192
	prefill 2 / decode 2	95.8	68.3	788	192
	prefill 4 / decode 2	94.3	69.6	788	192
MATH-500 Ouro-2.6B-T	native 2/2	85.0	243	3191	40
	native 4/4	82.5	469	3069	40
	prefill 4 / decode 2	85.0	298	3519	40
GSM8K SenseNova	native 2/2	98.4	48.0	365	64
	prefill 1 / decode 1	96.9	28.4	380	64
	prefill 2 / decode 1	95.3	30.6	380	64

Table 12: **End to end per answer**, prefill included (batch 1, one B300; mean seconds and generated tokens per answer over N problems; GSM8K: consecutive problem IDs 1–192; budgets 4,096 and 8,192 tokens). On GSM8K, prefill 4 / decode 2 spends $1.38\times$ the decode time per generated token of native 2/2 (pooled time over pooled tokens), but its answers are shorter.

8K context, 32 steps	n=1	n=2	n=3	n=4
native n/n	834	342	200	125
prefill 4 / decode n, one prompt cache	813	406	271	203
1K context, 2,048 steps		n=2		n=4
native n/n		586		219
prefill 4 / decode n, one prompt cache		570		253

Table 13: **Decode throughput** of Ouro-2.6B (tokens/s, best over tested batch sizes, one B300): native model against the loop adapter prefilling with 4 loops with one stored prompt KV cache.

I Design Variants

variant	m/n	task	difference
maps frozen at identity[†]	4/1	RULER-16K	-11.4 [-14.7, -8.2]
	4/1	GSM8K	$+2.3$ [0.2, 4.5]
	4/1	MMLU	-0.2 [-0.6, 0.1]
	4/2	RULER-16K	-2.6 [-4.5, -0.7]
	4/2	GSM8K	$+0.2$ [-1.3, 1.7]
	2/2	RULER-16K	-3.4 [-5.6, -1.3]
unrestricted keys[‡]	1/1	RULER-16K	$+0.1$ [-1.3, 1.5]
	2/2	RULER-16K	$+0.6$ [-0.8, 2.0]
	3/1	RULER-16K	-0.9 [-2.9, 1.2]
	3/2	RULER-16K	$+1.2$ [-0.4, 2.7]
	4/2	RULER-16K	-1.0 [-2.6, 0.5]

Table 14: **Design variants** on Ouro-2.6B: variant minus our loop adapter at prefill m / decode n (full sets, paired). Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals. [†]Same data, budget, seed and (m, n) sampling; only the maps are not trained (LoRA only, the decoder reads the last prefill loop's KV directly). [‡]Unrestricted key maps with one prompt KV cache per decode loop.

Table 14 compares design variants. Training the same loop adapter with its maps frozen at the identity, so that a LoRA decoder reads the last prefill loop's KV directly, changes MMLU by at most 0.4 points and is not worse on GSM8K ($+2.3$ [0.2, 4.5] at prefill 4 / decode 1), but lowers RULER-16K by 11.4 points at prefill 4 / decode 1 and by 2.6–3.4 points at two decode loops. Restricting the key map to a RoPE-commuting scale, which is what allows one stored prompt KV cache, shows no significant difference against unrestricted key maps with one cache per loop in five tested configurations (RULER-16K; Table 14).

Other design variants. Sharing generated-token KV across decode loops ("shared answer KV", Appendix A) costs 2.4 to 3.0 GSM8K points (-2.5 [-4.5, -0.5], -2.4 [-4.2, -0.7], -3.0 [-4.8, -1.3] at 4/2, 3/3, 4/4). A gated branch over the first prefill loop's KV improves RULER-4K by 2.3 [1.0, 3.6] points at prefill 4 / decode 1 with no significant difference on GSM8K ($+0.5$ [-0.9, 2.0]).

J Looped Decision Models: Setup and Checks

We evaluate on the 3,270 answerable questions of LevBench's public development set (never its locked test set), with intervals over record groups as Lev does. Each prefill count m is one continued training (seed 0); the main comparison (m equal to the backbone's loops) was repeated with three more seeds and against a control trained for twice as many updates. With all decision loops, the model with the most prefill loops is no better than the control on the Ouro backbones and below it on Lev-3B (-1.8 [-3.0, -0.7]); with $m = n$ the trained models are within a few points of the control in both directions (Table 15).

Setup and checks. Both arms continue each released Lev model for 140 updates on the 8,925-record public S31 training subset with Lev's hyperparameters, from the same weights. Our reference forward runs the full sequence through both the prefill and the decision loops, so its training computation is about twice the control's. Without training, more prefill loops do not help (-3.0 on Lev-1.4B at one decision loop), and the control itself moves little against the released models at one loop ($+0.5$ on Lev-1.4B, $+1.3$ on Lev-2.6B). Repeating both arms with other seeds, the one-loop gain ranges from $+3.9$ to $+5.3$ on Lev-1.4B and from $+5.1$ to $+7.5$ on Lev-2.6B, every interval excluding zero (four seeds each); on Lev-3B it is $+4.4$, $+5.1$, $+4.0$ and $+0.9$ [-0.8, 2.7] (mean $+3.6$). A control trained for two epochs (twice the updates, about the training computation of the other arm) still trails it at one decision loop by $+4.0$ [2.1, 6.0] on Lev-1.4B, $+5.5$ [3.7, 7.4] on Lev-2.6B and $+4.0$ [2.3, 5.7] on Lev-3B.

K Prefill Loops in a Hybrid Model

SenseNova-Loop-9B stacks three Gated-DeltaNet layers per full-attention layer. Its prompt state therefore has two parts: the attention KV, which the decoder attends to, and each Gated-DeltaNet layer's convolution and recurrent state, a fixed-size summary of the prompt from which the decoder continues and which it keeps updating with its own tokens (the prefill stops before the last prompt token, which the decoder processes first). We give the one-visit decoder the second visit's state for one kind of layer and the first visit's for the other (Table 16). The loop adapter was trained only with both states from the first or both from the second visit, so we also retrain it, with the same recipe and seed, giving the decoder the second visit's state only in the Gated-DeltaNet layers for prefill-2 samples (below). These are experiments on one checkpoint; they show how the decoder responds to each state, not where information is stored. In every condition the layers after the repeated block keep the two-visit prefill's states, so a comparison with

model	prefill m	decision 1	decision 2	decision 3	decision 4
Lev-1.4B	1	$+2.5^{*}$ [1.3, 3.7]	-2.8^{*} [−4.2, −1.4]	-6.1^{*} [−7.8, −4.6]	-9.6^{*} [−11.2, −7.9]
Lev-1.4B	2	$+2.9^{*}$ [1.3, 4.7]	$+2.6^{*}$ [1.4, 3.8]	-0.1 [−1.3, 1.1]	-2.7^{*} [−4.0, −1.4]
Lev-1.4B	3	$+5.7^{*}$ [3.9, 7.6]	$+3.8^{*}$ [2.3, 5.1]	$+1.8^{*}$ [0.7, 3.0]	-1.2^{*} [−2.4, −0.03]
Lev-1.4B	4	$+5.3^{*}$ [3.4, 7.2]	$+3.8^{*}$ [2.3, 5.3]	$+2.0^{*}$ [0.7, 3.2]	-0.1 [−1.3, 1.1]
Lev-2.6B	1	$+0.7$ [−0.5, 2.0]	-7.5^{*} [−9.1, −5.9]	-12.5^{*} [−14.1, −10.8]	-13.0^{*} [−14.7, −11.3]
Lev-2.6B	2	$+4.4^{*}$ [2.6, 6.2]	-0.8 [−2.0, 0.4]	-3.5^{*} [−4.8, −2.3]	-4.1^{*} [−5.5, −2.8]
Lev-2.6B	3	$+6.8^{*}$ [4.8, 8.7]	$+2.3^{*}$ [0.9, 3.8]	-0.4 [−1.5, 0.7]	-0.9 [−2.0, 0.3]
Lev-2.6B	4	$+7.5^{*}$ [5.7, 9.5]	$+3.5^{*}$ [2.0, 5.0]	$+0.5$ [−0.7, 1.7]	-0.3 [−1.4, 0.8]
Lev-3B	1	$+0.3$ [−0.7, 1.3]	-8.1^{*} [−9.7, −6.6]	–	–
Lev-3B	2	$+4.4^{*}$ [2.8, 6.1]	-1.8^{*} [−3.0, −0.7]	–	–

Table 15: **Lev decision models, both directions.** Accuracy of the model trained to prefill the context with m loops minus the matched-data and matched-update control (native n/n) at n decision loops, LevBench development set (3,270 answerable questions), in points, training seed 0 (other seeds in the text); brackets are 95% bootstrap intervals over record groups (*: interval excludes zero). Lev-3B's backbone has two loops. Above the diagonal ($m < n$) every difference is negative.

	untrained one-visit decoder			trained loop adapter		
benchmark	attention KV	GDN state	both	attention KV	GDN state	both
RULER-4K	$+41.3$ [37.2, 45.5]	$+48.0$ [44.0, 52.0]	$+61.9$ [57.8, 65.8]	-4.5 [−7.2, −2.0]	$+2.7$ [−0.5, 6.0]	-3.8 [−7.6, 0.2]
RULER-8K	$+49.0$ [44.8, 52.9]	$+49.6$ [45.7, 53.7]	$+64.2$ [60.0, 68.3]	-2.6 [−5.1, −0.2]	$+2.3$ [−0.8, 5.6]	-1.8 [−4.8, 1.3]
RULER-16K	$+50.2$ [46.0, 54.1]	$+50.5$ [46.4, 54.6]	$+66.6$ [62.7, 70.5]	$+0.7$ [−1.4, 2.8]	$+4.9$ [2.1, 7.7]	$+1.2$ [−1.7, 4.0]
MMLU-Pro	$+23.7$ [20.3, 27.0]	$+21.5$ [18.4, 24.8]	$+36.2$ [32.8, 39.8]	$+4.6$ [2.0, 7.2]	$+2.0$ [−0.4, 4.5]	$+5.5$ [2.9, 8.1]
ARC-C	$+5.2$ [3.2, 7.3]	$+7.7$ [4.9, 10.4]	$+9.6$ [6.7, 12.5]	$+0.8$ [−0.6, 2.1]	-2.7 [−4.5, −0.9]	-2.9 [−4.8, −1.0]
HELMET	$+32.1$ [30.1, 34.1]	$+37.4$ [35.5, 39.3]	$+51.8$ [49.8, 54.0]	-0.2 [−2.7, 2.2]	$+9.1$ [6.5, 11.6]	$+3.5$ [0.8, 6.1]

Table 16: **Mixed prompt-state configurations after a two-visit prefill.** SenseNova-Loop-9B, one decode visit, after a two-visit prefill. Within the repeated block, "attention KV" gives the decoder the second visit's attention KV and the first visit's Gated-DeltaNet (GDN) convolution and recurrent states; "GDN state" the reverse; "both" the second visit's states throughout. Layers outside the block keep the two-visit prefill's states. Differences are to the one-visit prefill with the same decoder, in score points; brackets are paired 95% bootstrap intervals.

the one-visit prefill changes those states too; only the comparisons among the three conditions hold them fixed.

Without training, both mixed-state conditions score far above the one-visit prefill (RULER-16K $+50.2$ and $+50.5$, against $+66.6$ for the full second-visit state; MMLU-Pro $+23.7$ and $+21.5$ against $+36.2$); these contrasts also change the post-block states, so they do not isolate either part. With the trained loop adapter the two parts behave differently. The condition with the second-visit Gated-DeltaNet state is higher on long-context scores: HELMET $+9.1$ [6.5, 11.6] and RULER-16K $+4.9$ [2.1, 7.7], against -0.2 and $+0.7$ for the second-visit attention KV, which lowers RULER-4K (-4.5 [−7.2, −2.0]). The mixed attention-KV condition is higher on MMLU-Pro ($+4.6$ [2.0, 7.2]; mixed Gated-DeltaNet condition $+2.0$ [−0.4, 4.5]). The ARC-Challenge loss of the full second-visit state (Table 17) appears with the Gated-DeltaNet part (-2.7 [−4.5, −0.9]; mixed attention-KV condition $+0.8$). In the "GDN state" condition the decoder still attends to the first visit's KV, consistent with exact retrieval relying on attention in hybrid models (Afendulev et al., 2026); what differs is the recurrent state it continues from (and the post-block states, see above). On Ouro, which has only attention layers, KV from more prefill loops likewise helps knowledge and loses exact recall (Appendix L). The same trained loop adapter, given the second-visit state only in its Gated-DeltaNet layers, scores above the full second-visit state on HELMET by 5.6 points and on RULER-4K/8K/16K by 6.5, 4.2 and 3.7, shows no detectable difference on ARC-Challenge, and scores below it on MMLU-Pro by 3.5. The loop adapter retrained on this mixed state confirms the pattern within its own training distribution: against its own prefill 1 / decode 1 it gains on HELMET ($+7.2$ [4.7, 9.7]) and RULER-16K ($+6.1$ [3.2, 9.1]) and loses ARC-Challenge (-2.8 [−4.7, −0.9]); against the original loop adapter at prefill 2 / decode 1 it is better on HELMET ($+4.4$ [2.7, 6.2]) and RULER-4K/8K/16K ($+6.7$, $+2.8$, $+2.9$) and worse on MMLU-Pro (-2.7 [−5.0, −0.3]). Which state to give the decoder

is therefore a task-dependent trade-off that holds with and without training for it.

Table 17 is the SenseNova-Loop-9B counterpart of Figure 4, at one decode visit through the repeated block instead of two. With the same loop adapter, prefilling with two visits instead of one raises MMLU-Pro ($+5.5$), BBH ($+7.0$) and HELMET ($+3.5$), lowers ARC-Challenge (-2.9), and leaves GSM8K, MATH-500 and HumanEval within noise. Against native 2/2, prefill 2 / decode 1 shows no detectable difference on GSM8K (-0.6 [-1.6, 0.4]; the $0.59\times$ answer time of Section 4.4 is measured for prefill 1 / decode 1) and scores lower on the other benchmarks, most on HELMET (-16.8); giving the decoder the second-visit state only in the Gated-DeltaNet layers narrows that HELMET gap to -11.2 [-13.0, -9.4]. The native one-visit model scores far lower on every benchmark.

L Prompt Format and HELMET Categories

HELMET shows which kinds of long-context task depend on the prefill loops (Table 18). On both Ouro reasoning models, prefilling with four loops instead of one improves retrieval-augmented QA by 15 points and re-ranking by 13–18, but lowers exact recall from a JSON key–value store on Ouro-1.4B-Thinking (-23.5); against native 4/4, prefill 4 / decode 2 has higher point estimates on retrieval-augmented QA and in-context learning ($+6.1$ [4.1, 8.4], $+6.6$ [3.0, 10.5]) and loses exact recall (58.8 vs. 100), so its HELMET average is lower (-8.7 [-13.3, -4.1]). SenseNova-Loop-9B does not follow the Ouro pattern on recall and re-ranking. The recall category is small (34 items on Ouro).

The effect of the prefill loops depends on the prompt format (Table 19). At one decode loop, more prefill loops give a negative but inconclusive estimate on RULER-4K, where answers are copied verbatim, and lower 4-shot MATH-500 on both model sizes; without exemplars the same comparison turns positive on two model sizes and two decode loop counts. Removing exemplars also changes length and format, so we read this as an interaction with the prompt.

	score				prefill 2 / decode 1 minus	
benchmark	native 1/1	prefill 1 / decode 1	prefill 2 / decode 1	native 2/2	prefill 1 / decode 1	native 2/2
GSM8K	2.2	95.6	95.2	95.8	-0.4 [-1.4, 0.6]	-0.6 [-1.6, 0.4]
MATH-500	3.2	88.0	89.8	94.8	$+1.8$ [-1.0, 4.6]	-5.0 [-7.2, -2.8]
MMLU-Pro	18.9	60.3	65.8	68.2	$+5.5$ [2.9, 8.1]	-2.3 [-4.4, -0.2]
ARC-C	52.4	66.0	63.1	66.9	-2.9 [-4.8, -1.0]	-3.8 [-5.3, -2.4]
BBH	2.6	44.5	51.5	57.6	$+7.0$ [5.9, 8.0]	-6.1 [-7.0, -5.2]
HumanEval	3.0	75.0	76.2	81.7	$+1.2$ [-4.9, 7.3]	-5.5 [-11.0, 0.0]
HELMET	7.2	56.9	60.3	77.1	$+3.5$ [0.8, 6.1]	-16.8 [-18.7, -15.0]

Table 17: **SenseNova-Loop-9B at one decode visit.** One loop adapter serves prefill 1 / decode 1 and prefill 2 / decode 1; native 2/2 runs the repeated block twice per token (56 layer executions instead of 32). Same protocols as Table 9 in Appendix F (MMLU-Pro 980, HELMET 6,646, BBH 6,511 items). MBPP+ is omitted: its prompt forces the answer to start with code, and this thinking model then opens a reasoning block inside the code (native 1/1: 294, prefill 1 / decode 1: 236, prefill 2 / decode 1: 372, native 2/2: 375 of 378 answers), so its score measures this format failure. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

comparison	RAG	re-rank	ICL	recall
2.6B-T, prefill $4-1$	$+15.4$	$+13.4$	$+5.1$	-2.9
1.4B-T, prefill $4-1$	$+14.5$	$+18.0$	$+9.3$	-23.5
SenseNova, prefill $2-1$	$+1.2$	-32.9	$+21.6$	$+24.0$
2.6B-T, 4/2 − native 4/4	$+6.1$	-6.3	$+6.6$	-41.2

Table 18: **HELMET by category** (score differences on a 0–100 scale; prefill comparisons at one decode loop, same loop adapter). More prefill loops help retrieval-augmented QA (RAG), re-ranking and in-context learning (ICL) on Ouro but can lose exact key–value recall. Items: 700 / 41 / 333 / 34 per category on Ouro, 4,200 / 246 / 2,000 / 200 on SenseNova; intervals are in `paper_numbers.json`.

model	decode n	task	prefill $4-$ prefill n
Ouro-2.6B	1	RULER-4K	-3.8 [-8.2, 0.3]
	1	MATH, 4-shot	-12.0 [-16.2, -7.8]
	1	MATH, 0-shot	$+8.0$ [4.2, 11.6]
	2	MATH, 0-shot	$+7.4$ [3.4, 11.4]
Ouro-1.4B	1	MATH, 4-shot	-4.4 [-8.4, -0.6]
	1	MATH, 0-shot	$+4.8$ [1.2, 8.4]

Table 19: **The effect of prefill loops changes with the prompt setup.** Same loop adapter, base models, prefill 4 / decode n minus prefill n / decode n; MATH-500 scored on the final answer. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

M Reproducibility

`python ouro/scripts/paper_numbers.py compute` recomputes the paired differences, intervals, means, timing ratios and decode throughput of this paper from the accompanying per-item records and compares them with expected values hard-coded in the script, which mirror the numbers printed here; only the MBPP+ format counts of Appendix B are reported as measured. Every run's command, configuration, checkpoint hash, log and outputs are in `ouro/results/provenance`. Re-running the main-table cells from scratch reproduces GSM8K and MMLU with at least 99.6% of item scores identical and RULER-16K with 94–100% identical and averages within 0.9 points (`ouro/results/verify_rerun.txt`). `REPRODUCE.md` maps every result to its scripts and gives the training and evaluation commands that regenerate the per-item records.